GUIDE D'INSTALLATION ET D'UTILISATION

SignageCMS

Serveur d'affichage dynamique

Installation pas à pas du serveur sous Windows et Linux, première connexion, création des médias et des présentations, planification de la diffusion et connexion des lecteurs web et Android — illustré par les écrans réels du produit.

Documentation produit

Windows 10 / 11 · Windows Server · Ubuntu · Debian · Lecteur web · Android

Table des matières

1. Présentation générale 1

2. Configuration requise 3

3. Installation sous Windows 4

4. Installation sous Linux 6

5. Premier démarrage et première connexion 23

6. Prise en main de l'interface 25

7. Écrans : création et connexion des lecteurs 27

8. Médias, playlists et présentations 29

9. Campagnes 32

10. Planification de la diffusion 35

11. Le lecteur web et l'application Android 37

12. Maintenance, sécurité et dépannage 41

Annexe A — Référence rapide 43

1. Présentation générale

SignageCMS est un système complet de gestion d'affichage dynamique (*digital signage*) auto-hébergé. Il se compose d'un serveur web unique qui centralise la bibliothèque de médias, la construction des présentations, les campagnes, la planification et les réglages, ainsi que de lecteurs légers installés sur les écrans : un lecteur web intégré (page plein écran pour navigateur, box TV ou mini-PC) et une application Android dédiée. Toute l'administration se fait depuis un simple navigateur, sans logiciel à installer sur les postes de gestion, et la base de données embarquée (SQLite) évite tout serveur SQL externe.

La chaîne de travail reste identique quel que soit le lecteur : vous importez des **médias** (images, vidéos, sons, PowerPoint, PDF), vous les assemblez en **playlists** et en **présentations** (mises en page par régions et widgets), vous regroupez éventuellement plusieurs présentations en **campagnes**, puis vous créez des **événements de planification** qui programment quoi afficher, quand et sur quels écrans. Chaque écran est identifié par une **clé matérielle** unique : c'est elle qui relie le lecteur au CMS, sans compte utilisateur à saisir sur les écrans.

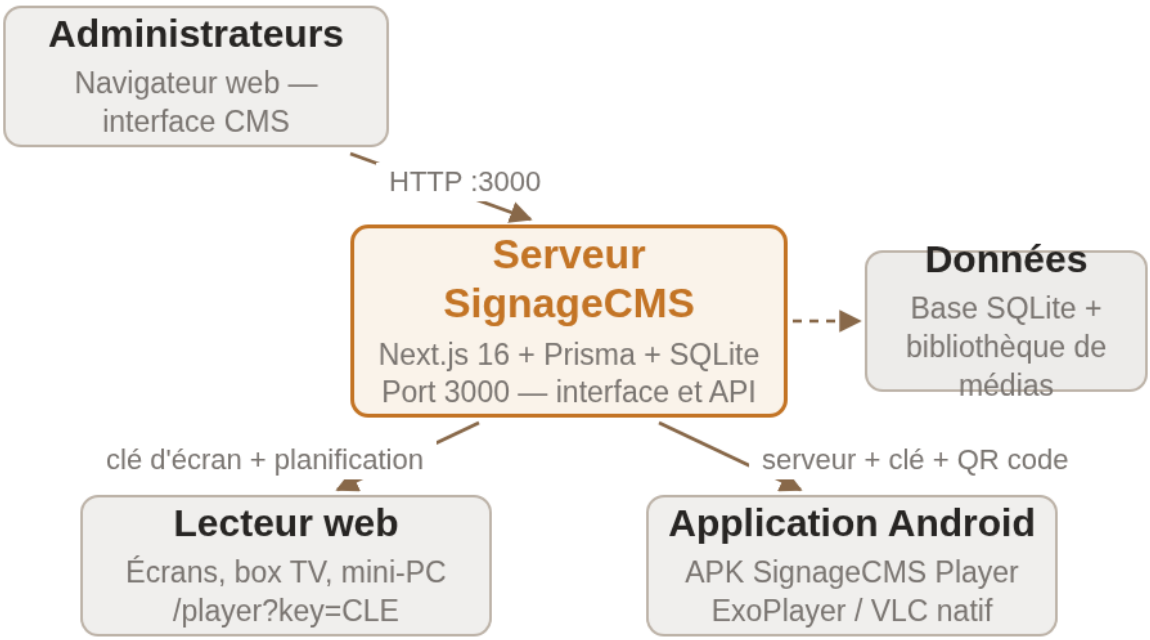


Figure 1 — Architecture générale : un serveur, plusieurs types de lecteurs

Le schéma ci-dessus résume les échanges : les administrateurs pilotent le serveur depuis leur navigateur sur le port 3000 ; les lecteurs web et l'application Android interrogent le même serveur pour récupérer le contenu planifié, en s'authentifiant uniquement avec la clé d'écran. Le serveur stocke la base (comptes, planifications, réglages) et les médias téléversés dans son dossier de données.

Fonctionnalités principales

- **Écrans et groupes** : inventaire des lecteurs, statut en ligne / hors ligne par battements de cœur, aperçu à distance, groupes de diffusion.
- **Bibliothèque de médias** : images, vidéos, audio, PDF, PowerPoint (conversion automatique en diapositives), données JSON/CSV/XLSX.
- **Présentations (layouts)** : concepteur visuel par régions avec widgets média, playlist, texte défilant, horloge, météo, page web, IPTV, compte à rebours, RSS.
- **Campagnes** : enchaînement de plusieurs présentations, avec durée par élément et boucle paramétrable.
- **Planification** : calendrier hebdomadaire, fenêtres horaires, cibles multiples (tous les écrans, groupes, liste), boucle ON/OFF par événement.
- **TV / IPTV** : sources Xtream et listes M3U, proxy intégré, partage intelligent des connexions, pool multi-comptes, timeshift.
- **Pages web interactives** : affichage de sites même anti-iframe grâce au proxy de pages intégré (protection anti-SSRF).
- **Rapports** : journaux serveur et preuves de diffusion (proof-of-play) exportables.
- **Comptes et droits** : rôles admin / user / read-only, matrice de permissions par module, import LDAP / Active Directory.
- **Sécurité** : mot de passe admin généré aléatoirement (aucun mot de passe par défaut), limiteur de tentatives, durcissement des téléversements.

Organisation de ce guide

Les chapitres 2 à 4 couvrent les prérequis et l'installation pas à pas du serveur, d'abord sous Windows (installateur graphique assisté) puis sous Linux (lignes de commande et service systemd). Les chapitres 5 et 6 accompagnent le premier démarrage et la découverte de l'interface d'administration. Les chapitres 7 à 11 forment le guide d'utilisation proprement dit : écrans, médias, présentations, campagnes, planification et lecteurs. Le chapitre 12 rassemble la maintenance, les réflexes de sécurité et les solutions aux incidents courants, et l'annexe A offre une référence rapide des commandes, fichiers et liens. Toutes les captures d'écran de ce guide proviennent d'une instance réelle du serveur.

2. Configuration requise

SignageCMS est volontairement sobre : un seul processus Node.js sert l'interface d'administration, l'API des lecteurs et les fichiers médias. Une machine modeste suffit pour un parc de quelques dizaines d'écrans ; les charges lourdes (nombreux flux IPTV simultanés, vidéos haute définition) tirent surtout sur le réseau et la bande passante. Le tableau ci-dessous récapitule les besoins pour chaque plateforme d'hébergement.

Élément	Windows	Linux
Système	Windows 10 / 11 ou Windows Server 2016 et plus récent (64 bits)	Ubuntu 20.04+, Debian 11+, Fedora, RHEL, openSUSE, Arch ou Alpine
Mémoire vive	4 Go minimum, 8 Go recommandés	2 Go minimum, 4 Go recommandés
Disque	5 Go libres + espace médias (estimer 1 à 5 Go par année de diffusion)	Idem : 5 Go libres + espace médias
Runtime	Node.js 20 LTS ou 22 LTS (obligatoire, 20.9 minimum)	Node.js 20 LTS / 22 LTS, ou Bun 1.x
Port réseau	3000 (TCP) entrant, modifiable via la variable PORT	3000 (TCP) entrant, modifiable via la variable PORT
Conversion PPT / PDF	LibreOffice + Poppler (installés automatiquement par l'installateur)	LibreOffice + Poppler (inclus dans le script install.sh — installation complète)

Deux points méritent une attention particulière au moment du choix du serveur. D'une part, la version de Node.js conditionne la compilation : Next.js 16 exige Node.js 20.9 au minimum, et l'installateur Windows refuse poliment de continuer si le prérequis manque. D'autre part, les outils de conversion (LibreOffice pour transformer les PowerPoint en PDF, Poppler pour rasteriser les PDF en images) sont **optionnels** : le CMS démarre et fonctionne sans eux, seuls les documents bureautiques ne sont pas convertis en diapositives diffusables.

> **Écrans côté diffusion**
>
> Chaque écran a uniquement besoin d'un navigateur récent (Chrome, Edge, Firefox) pour le lecteur web, ou d'Android 7.0 minimum pour l'application dédiée. Les téléviseurs connectés avec navigateur intégré, les box TV, les mini-PC sous Windows ou Linux conviennent également. Aucun compte n'est saisi sur l'écran : seule la clé matérielle de l'écran est nécessaire.

Côté réseau, le serveur et les écrans doivent pouvoir communiquer sur le port 3000 en TCP. Dans un réseau local d'entreprise, vérifiez que le pare-feu du serveur autorise ce port et que les VLAN d'écrans accèdent au VLAN du serveur. Si le CMS doit être exposé au-delà du réseau local (multi-sites, accès Internet), l'usage d'un reverse proxy HTTPS tel que Caddy ou Nginx est fortement recommandé ; la procédure figure au chapitre 4.

3. Installation sous Windows

Vue d'ensemble de la procédure

L'installation Windows repose sur des scripts assistés livrés dans le dossier **deploy\windows** de l'archive. La séquence officielle est la suivante : **install.bat** (installation complète) puis **start-cms.bat** (démarrage du serveur), **firewall.ps1** (ouverture du port réseau) et enfin **install-service.ps1** (démarrage automatique au boot). L'installateur crée le fichier .env, installe les dépendances, prépare la base SQLite, compile la version production et télécharge les outils de conversion PowerPoint/PDF. Comptez 10 à 20 minutes au total selon la connexion, dont la majorité pour les dépendances et le téléchargement de LibreOffice.

Étape 1 — Installer Node.js

1. Rendez-vous sur **https://nodejs.org** et téléchargez la version **20 LTS** ou **22 LTS** (installateur .msi 64 bits).
2. Exécutez l'installateur en laissant toutes les options par défaut (le chemin PATH est coché automatiquement).
3. Ouvrez une console (cmd) et vérifiez : **node -v** doit afficher v20.x ou v22.x.

Étape 2 — Récupérer et décompresser l'archive serveur

Téléchargez l'archive **SignageCMS-Serveur.zip** depuis le lien de l'annexe A, puis décompressez-la à l'emplacement de votre choix. Les chemins contenant des espaces ou des parenthèses sont acceptés (par exemple C:\Program Files\SignageCMS), mais un chemin court comme **C:\SignageCMS** facilite les opérations quotidiennes. Après décompression, vous devez obtenir un dossier **signagecms** contenant les sous-dossiers src, prisma, public, db et deploy.

Étape 3 — Lancer l'installation automatique

1. Ouvrez le dossier décompressé et double-cliquez sur **deploy\windows\install.bat**.
2. Laissez le script travailler : il vérifie Node.js, crée le fichier .env avec le chemin réel de la base, exécute npm install (5 à 10 minutes), prépare la base avec Prisma, puis compile la version production (2 à 6 minutes).
3. À l'étape 6, l'installateur télécharge et installe **LibreOffice** (MSI officiel, environ 360 Mo) et **Poppler** pour la lecture des PowerPoint et PDF. Pour ignorer cette étape : tapez **set SKIP_TOOLS=1** dans la console avant de relancer le script.
4. En cas de souci, consultez les journaux **deploy\windows\install-cms.log**, **build-cms.log** et **install-tools.log**.

Si la console n'a pas les droits administrateur, l'installateur termine quand même et vous invite à exécuter plus tard **install-tools.bat** en mode « Exécuter en tant qu'administrateur » pour installer les outils de conversion. Ce script est réexécutable sans risque : il détecte ce qui est déjà présent et ne réinstalle rien inutilement.

Étape 4 — Démarrer le serveur

1. Double-cliquez sur **deploy\windows\start-cms.bat**.
2. Attendez le message de démarrage : le serveur écoute sur **http://localhost:3000** et sur toutes les interfaces réseau.
3. Au premier lancement, le serveur crée le compte administrateur et affiche son **mot de passe aléatoire dans la console**, puis l'enregistre dans **db\admin-credentials.txt**.

Étape 5 — Autoriser l'accès depuis le réseau

Par défaut, le pare-feu Windows bloque les connexions entrantes : les écrans du réseau ne peuvent donc pas joindre le serveur. Faites un clic droit sur **deploy\windows\firewall.ps1** puis « Exécuter avec PowerShell » en tant qu'administrateur : une règle d'entrée ouvre le port 3000 en TCP. Les lecteurs peuvent alors pointer vers **http://ADRESSE-IP-DU-SERVEUR:3000**. Utilisez la commande ipconfig pour connaître l'adresse IP réelle du serveur ; le CMS affiche aussi cette adresse dans le dialogue « Connecter Android » (chapitre 7).

Étape 6 — Démarrage automatique au boot

Pour que le serveur redémarre tout seul après un redémarrage de la machine (et en cas de crash), exécutez **deploy\windows\install-service.ps1** en tant qu'administrateur. Le script installe un service Windows nommé **SignageCMS** : il utilise NSSM si l'outil est disponible, sinon il crée une tâche planifiée SYSTEM qui relance start-cms.bat. La désinstallation se fait avec **nssm remove SignageCMS confirm** ou **schtasks /Delete /TN SignageCMS /F**. Le fonctionnement du service se vérifie dans le fichier **service.log** à la racine du projet et avec Get-Service SignageCMS dans PowerShell.

Installation manuelle (scripts bloqués par la stratégie)

En environnement verrouillé (GPO, antivirus), exécutez les commandes suivantes depuis une console PowerShell ouverte à la racine du projet. Le résultat est identique à l'installation automatique, à l'exception des outils de conversion qu'il faut alors installer séparément (chapitre 8).

```
cd C:\SignageCMS
npm install
npx prisma db push
npx prisma generate
npm run build
node .next\standalone\server.js
```

Récapitulatif des scripts Windows

Script	Rôle
install.bat	Installation complète : vérification Node.js, .env, dépendances, base, build, outils de conversion. Point d'entrée unique.
start-cms.bat	Démarrage en production sur le port 3000 (surchargeable : set PORT=8080). Écoute sur toutes les interfaces.
build.bat	Recompilation seule après une mise à jour du code (déjà appelé par install.bat).
firewall.ps1	Ouvre le port 3000 du pare-feu Windows (à exécuter en administrateur).
install-service.ps1	Installe le service Windows auto-démarré (NSSM ou tâche planifiée SYSTEM).
install-tools.bat	Télécharge et installe LibreOffice + Poppler depuis les sources officielles. Réexécutable sans risque.
dev.bat	Mode développement avec rechargement à chaud — à ne pas utiliser en production.

4. Installation sous Linux

Le chapitre propose **deux chemins** vers le même résultat. Le chemin recommandé est le script **install.sh** livré dans le dossier deploy/linux de l'archive : une seule commande enchaîne tout — utilitaires, Node.js 20 LTS, LibreOffice et Poppler, déploiement, compilation, service systemd et pare-feu. Le chemin manuel, détaillé ensuite section par section, s'adresse aux administrateurs qui veulent contrôler chaque étape ou s'intégrer à une procédure interne existante (Ansible, image dorée, conteneur).

Prérequis système

La procédure ci-dessous est rédigée pour Ubuntu 22.04 / Debian 12 avec Node.js ; elle se transpose directement aux autres distributions (seules les commandes d'installation des paquets changent). Connectez-vous au serveur en SSH, obtenez les droits administrateur et installez les utilitaires de base ainsi que Node.js 20 LTS depuis le dépôt officiel NodeSource. Une alternative moderne consiste à utiliser **Bun**, plus rapide, avec la commande d'installation officielle indiquée en commentaire.

```
sudo apt update && sudo apt install -y curl unzip ca-certificates

# Node.js 20 LTS (NodeSource) :
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node -v # doit afficher v20.x ou plus récent

# Variante Bun (optionnelle) :
# curl -fsSL https://bun.sh/install | bash
```

Installation automatisée complète — le script install.sh (recommandé)

Le fichier **deploy/linux/install.sh**, livré dans l'archive serveur (et également téléchargeable séparément, lien en annexe A), réalise l'intégralité de l'installation d'un serveur CMS opérationnel : il installe d'abord les utilitaires de base, puis Node.js 20 LTS s'il est absent, puis **LibreOffice et Poppler** nécessaires à la conversion des PowerPoint et PDF, déploie les sources dans /opt/signagecms, crée le fichier .env, prépare la base Prisma, compile la version production, installe un service systemd avec utilisateur dédié, ouvre le port dans le pare-feu et termine par une vérification de l'API. Le script détecte seul votre distribution (Ubuntu / Debian, RHEL / Fedora / Rocky / Alma, openSUSE, Arch, Alpine) et est **idempotent** : réexécutable sans risque, il ne réinstalle rien qui est déjà présent et conserve un fichier .env existant.

```
# archive SignageCMS-Serveur.zip téléchargée dans ~/Téléchargements :
cd ~/Téléchargements
sudo bash install.sh --zip ~/Téléchargements/SignageCMS-Serveur.zip

# ou après extraction de l'archive (le script trouve le zip à côté de lui) :
sudo bash /opt/signagecms/deploy/linux/install.sh

# options utiles :
sudo bash install.sh --port 8080 # port d'écoute du CMS
sudo bash install.sh --dir /opt/moncms # autre dossier d'installation
sudo bash install.sh --no-service # sans unité systemd (test, conteneur)
sudo bash install.sh --no-firewall # ne modifie pas ufw / firewalld
```

Étape	Ce que fait le script
1/8 — Utilitaires	Installe curl, unzip et les certificats CA (paquets officiels de la distribution).
2/8 — Node.js	Vérifie Node.js 18+ ; sinon installe Node.js 20 LTS (NodeSource sous apt/dnf, dépôts natifs sinon).
3/8 — LibreOffice + Poppler	Installe libreoffice-impress et poppler-utils si absents (conversion PPT → PDF → images).
4/8 — Déploiement	Extrait l'archive serveur dans /opt/signagecms (option --dir pour un autre chemin).
5/8 — Fichier .env	Crée DATABASE_URL et PORT ; conserve intact un .env existant.
6/8 — Build production	npm install, prisma db push + generate, npm run build (dossier .next/standalone).
7/8 — Service systemd	Crée l'utilisateur dédié signage, l'unité signagecms.service, active le démarrage au boot.
8/8 — Pare-feu + test	Ouvre le port (ufw ou firewalld), interroge /api/version jusqu'à réponse.

Le script complet est reproduit ci-dessous en quatre parties, telles qu'embarquées dans l'archive serveur — vous pouvez le copier depuis ce guide ou le télécharger directement (annexe A). Les parties sont séparées aux mêmes frontières que les commentaires de section du fichier.

Partie 1 — En-tête, options de ligne de commande, droits et détection de la distribution

```
#!/usr/bin/env bash
# ============================================================
# SignageCMS — INSTALLATION COMPLETE SOUS LINUX
# ------------------------------------------------------------
# Ce script automatise l'integralite de l'installation :
# 1. Utilitaires de base (curl, unzip, ca-certificates)
# 2. Node.js 20 LTS (depot officiel NodeSource)
# 3. LibreOffice + Poppler (conversion PowerPoint/PDF)
# 4. Depot de l'archive serveur dans /opt/signagecms
# 5. Fichier .env (base SQLite, port, options)
# 6. npm install + Prisma + build production
# 7. Service systemd (demarrage auto, restart apres crash)
# 8. Pare-feu (port 3000) + verification finale
#
# Usage :
# sudo bash install.sh
# sudo bash install.sh --zip /chemin/SignageCMS-Serveur.zip
# sudo bash install.sh --port 8080
# sudo bash install.sh --no-service (sans systemd)
# sudo bash install.sh --no-firewall (ne touche pas au pare-feu)
#
# Idempotent : reexecutable sans risque, ne reinstalle rien
# qui est deja present. Compatible Ubuntu / Debian (apt),
# RHEL / Fedora / Rocky / Alma (dnf/yum), openSUSE (zypper),
# Arch (pacman), Alpine (apk).
# ============================================================
set -u

APP_DIR="/opt/signagecms"
CMS_PORT="3000"
ZIP_PATH=""
WITH_SERVICE=1
WITH_FIREWALL=1

while [ $# -gt 0 ]; do
case "$1" in
--zip) ZIP_PATH="$2"; shift 2 ;;
--port) CMS_PORT="$2"; shift 2 ;;
--dir) APP_DIR="$2"; shift 2 ;;
--no-service) WITH_SERVICE=0; shift ;;
--no-firewall) WITH_FIREWALL=0; shift ;;
*)
echo "Option inconnue : $1"
echo "Options : --zip FICHIER --port N --dir CHEMIN --no-service --no-firewall"
```

```
exit 1 ;;
esac
done

# ---------- affichage ------------------------------------------------
ok() { printf '\033[32m[OK]\033[0m %s\n' "$*"; }
warn() { printf '\033[33m[!]\033[0m %s\n' "$*"; }
err() { printf '\033[31m[ERREUR]\033[0m %s\n' "$*"; }
info() { printf '\033[36m[..]\033[0m %s\n' "$*"; }
hr() { printf '\033[2m----------------------------------------------\033[0m\n'; }

echo "============================================================"
echo " SignageCMS - installation complete sous Linux"
echo "============================================================"
echo " Dossier cible : $APP_DIR"
echo " Port du CMS : $CMS_PORT"
hr

# ---------- droits administrateur ------------------------------------
# SUDO_USABLE=1 seulement si root, sudo sans mot de passe, ou terminal
# interactif (l'utilisateur pourra saisir son mot de passe). Sinon, le
# script continue en mode non privilegie : les etapes systeme (paquets,
# systemd, pare-feu) sont degradees proprement, et l'installation est
# possible des que le dossier cible est inscriptible par l'utilisateur.
SUDO=""
SUDO_USABLE=0
if [ "$(id -u)" -eq 0 ]; then
SUDO=""
SUDO_USABLE=1
elif command -v sudo >/dev/null 2>&1; then
SUDO="sudo"
if sudo -n true 2>/dev/null || [ -t 0 ]; then
SUDO_USABLE=1
info "Execution en tant qu'utilisateur courant via sudo."
else
warn "sudo demande un mot de passe (session non interactive)."
warn "Etapes paquets/systemd/pare-feu limitees."
warn "Installation directe tentee si le dossier est inscriptible."
fi
else
warn "sudo absent - installation non privilegiee si possible."
fi

# ---------- gestionnaire de paquets ----------------------------------
PKG=""
for m in apt-get dnf yum zypper pacman apk; do
```

```
command -v "$m" >/dev/null 2>&1 && { PKG="$m"; break; }
done
if [ -z "$PKG" ]; then
err "Aucun gestionnaire de paquets connu (apt/dnf/yum/zypper/pacman/apk)."
exit 1
fi
ok "Gestionnaire de paquets : $PKG"

pkg_install() {
if [ "$SUDO_USABLE" -eq 0 ]; then
warn "Paquets non installes (root requis) : $* - installez-les manuellement."
return 0
fi
case "$PKG" in
apt-get) $SUDO apt-get install -y "$@" ;;
dnf) $SUDO dnf install -y "$@" ;;
yum) $SUDO yum install -y "$@" ;;
zypper) $SUDO zypper --non-interactive install "$@" ;;
pacman) $SUDO pacman -S --noconfirm "$@" ;;
apk) $SUDO apk add "$@" ;;
esac
}
pkg_refresh() {
[ "$SUDO_USABLE" -eq 1 ] || return 0
case "$PKG" in
apt-get) $SUDO apt-get update ;;
dnf) $SUDO dnf makecache --refresh >/dev/null 2>&1 || true ;;
yum) $SUDO yum makecache >/dev/null 2>&1 || true ;;
apk) $SUDO apk update >/dev/null 2>&1 || true ;;
esac
}

# ---------- 1. utilitaires de base ----------------------------------
hr
info "1/8 - Utilitaires de base (curl, unzip, ca-certificates)"
pkg_refresh
pkg_install curl unzip ca-certificates
ok "Utilitaires de base prets."
```

Partie 2 — Node.js 20 LTS puis LibreOffice et Poppler (conversion PowerPoint / PDF)

Node.js n'est installé que si une version 18 ou supérieure est absente ; sous Debian et Ubuntu le script passe par le dépôt officiel NodeSource, ailleurs par les dépôts natifs. La conversion des documents est gérée de la même façon : le choix des paquets s'adapte à la distribution (libreoffice-impress + poppler-utils, poppler-tools sur openSUSE, libreoffice-still + poppler sur Arch) et chaque outil déjà présent est simplement vérifié.

```
# ---------- 2. Node.js 20 LTS ----------------------------------------
hr
info "2/8 - Node.js 20 LTS"
NODE_OK=0
if command -v node >/dev/null 2>&1; then
NODE_V="$(node -v | sed 's/^v//' | cut -d. -f1)"
if [ "${NODE_V:-0}" -ge 18 ] 2>/dev/null; then
NODE_OK=1
ok "Node.js $(node -v) deja present (>= 18 requis)."
fi
fi
if [ "$NODE_OK" -eq 0 ]; then
case "$PKG" in
apt-get)
info "Installation depuis le depot NodeSource..."
curl -fsSL https://deb.nodesource.com/setup_20.x | $SUDO -E bash - ;;
dnf|yum)
info "Installation depuis le depot NodeSource..."
curl -fsSL https://rpm.nodesource.com/setup_20.x | bash - ;;
zypper) info "Installation nodejs20 depuis les depots openSUSE..." ;;
pacman) info "Installation nodejs/npm depuis les depots Arch..." ;;
apk) info "Installation nodejs/npm depuis les depots Alpine..." ;;
esac
case "$PKG" in
zypper) pkg_install nodejs20 ;;
pacman) pkg_install nodejs npm ;;
apk) pkg_install nodejs npm ;;
*) pkg_install nodejs ;;
esac
command -v node >/dev/null 2>&1 \
|| { err "Node.js absent apres installation - verifiez le depot."; exit 1; }
ok "Node.js $(node -v) installe."
fi
command -v npm >/dev/null 2>&1 || pkg_install npm

# ---------- 3. LibreOffice + Poppler ---------------------------------
hr
info "3/8 - LibreOffice + Poppler (conversion PowerPoint/PDF)"
LO_PKG=""; PP_PKG=""
case "$PKG" in
apt-get) LO_PKG="libreoffice-impress"; PP_PKG="poppler-utils" ;;
dnf|yum) LO_PKG="libreoffice-impress"; PP_PKG="poppler-utils" ;;
zypper) LO_PKG="libreoffice-impress"; PP_PKG="poppler-tools" ;;
pacman) LO_PKG="libreoffice-still"; PP_PKG="poppler" ;;
apk) LO_PKG="libreoffice"; PP_PKG="poppler-utils" ;;
esac
```

```
if command -v soffice >/dev/null 2>&1; then
ok "LibreOffice deja installe : \
$(timeout 10 soffice --version 2>/dev/null | head -1 || echo soffice)"
else
info "Installation de LibreOffice ($LO_PKG) - plusieurs minutes..."
pkg_install $LO_PKG
if command -v soffice >/dev/null 2>&1; then
ok "LibreOffice installe."
else
warn "soffice introuvable - les PowerPoint ne seront pas convertis."
fi
fi
if command -v pdftoppm >/dev/null 2>&1; then
ok "Poppler deja installe : $(timeout 5 pdftoppm -v 2>&1 | head -1)"
else
info "Installation de Poppler ($PP_PKG)..."
pkg_install $PP_PKG
if command -v pdftoppm >/dev/null 2>&1; then
ok "Poppler installe."
else
warn "pdftoppm introuvable - les PDF ne seront pas decoupes."
fi
fi
```

Partie 3 — Déploiement de l'archive, fichier .env et compilation production

Le script recherche l'archive SignageCMS-Serveur-*.zip à côté de lui puis dans le dossier courant (option --zip pour forcer un chemin). Si une installation existe déjà dans /opt/signagecms, l'archive devient optionnelle et sa fourniture déclenche une ré-extraction écrasante. Le fichier .env n'est jamais écrasé ; le build est lancé avec une réserve mémoire de 2 Go (NODE_OPTIONS) pour éviter toute interruption sur petite machine.

```
# ---------- 4. depot de l'archive serveur ----------------------------
hr
info "4/8 - Depot de l'archive serveur dans $APP_DIR"
SCRIPT_DIR="$(cd "$(dirname "$0")" 2>/dev/null && pwd)"
if [ -z "$ZIP_PATH" ]; then
if [ -n "$SCRIPT_DIR" ]; then
ZIP_PATH="$(ls "$SCRIPT_DIR"/../../SignageCMS-Serveur-*.zip \
"$SCRIPT_DIR"/SignageCMS-Serveur-*.zip 2>/dev/null | head -1)"
fi
[ -n "$ZIP_PATH" ] \
|| ZIP_PATH="$(ls "$PWD"/SignageCMS-Serveur-*.zip 2>/dev/null | head -1)"
fi
if [ -d "$APP_DIR/.next" ]; then
if [ -z "$ZIP_PATH" ]; then
```

```
ok "Installation existante detectee : $APP_DIR (archive non requise)."
else
info "Archive fournie : re-extraction ecrasante dans $APP_DIR..."
PARENT="$(dirname "$APP_DIR")"
$SUDO unzip -oq "$ZIP_PATH" -d "$PARENT" 2>/dev/null \
|| unzip -oq "$ZIP_PATH" -d "$PARENT" \
|| { err "Extraction impossible (droits sur $PARENT ?)."; exit 1; }
[ "$PARENT/signagecms" = "$APP_DIR" ] \
|| $SUDO mv -T "$PARENT/signagecms" "$APP_DIR" 2>/dev/null \
|| mv -T "$PARENT/signagecms" "$APP_DIR"
[ -d "$APP_DIR/src" ] \
|| { err "Re-extraction douteuse : $APP_DIR/src absent."; exit 1; }
ok "Archive re-extraite."
fi
else
if [ -z "$ZIP_PATH" ]; then
err "Archive zip introuvable."
echo " Utilisez : --zip /chemin/SignageCMS-Serveur.zip"
exit 1
fi
ok "Archive : $ZIP_PATH"
PARENT="$(dirname "$APP_DIR")"
$SUDO mkdir -p "$PARENT" 2>/dev/null || mkdir -p "$PARENT" \
|| { err "Creation de $PARENT impossible (droits ?)."; exit 1; }
$SUDO unzip -oq "$ZIP_PATH" -d "$PARENT" 2>/dev/null \
|| unzip -oq "$ZIP_PATH" -d "$PARENT" \
|| { err "Extraction impossible (droits sur $PARENT ?)."; exit 1; }
[ "$PARENT/signagecms" = "$APP_DIR" ] \
|| { $SUDO mv -T "$PARENT/signagecms" "$APP_DIR" 2>/dev/null \
|| mv -T "$PARENT/signagecms" "$APP_DIR"; }
[ -d "$APP_DIR/src" ] || {
err "Archive extraite mais $APP_DIR/src absent."
echo " Verifiez les droits ou l'integrite de l'archive."
exit 1
}
ok "Archive extraite dans $APP_DIR."
fi

# ---------- 5. fichier .env ----------------------------------------
hr
info "5/8 - Fichier .env (base SQLite, port)"
cd "$APP_DIR" || { err "Acces a $APP_DIR impossible."; exit 1; }
mkdir -p db uploads-tmp 2>/dev/null || $SUDO mkdir -p db uploads-tmp
if [ -f .env ]; then
warn ".env existant conserve (port et base inchanges)."
else
```

```
ENV_CONTENT="DATABASE_URL=file:$APP_DIR/db/custom.db
PORT=$CMS_PORT
# MAX_UPLOAD_MB=500
# SOFFICE_PATH=/chemin/vers/soffice
# ALLOW_FRAME_EMBED=1 (si integration dans une iframe externe)"
printf '%s\n' "$ENV_CONTENT" | $SUDO tee .env >/dev/null 2>/dev/null \
|| printf '%s\n' "$ENV_CONTENT" > .env \
|| { err "Ecriture de .env impossible (droits ?)."; exit 1; }
ok ".env cree (DATABASE_URL + PORT=$CMS_PORT)."
fi

# ---------- 6. dependances + Prisma + build -------------------------
hr
info "6/8 - Dependances npm, schema Prisma, build production"
$SUDO npm install --no-audit --no-fund 2>/dev/null \
|| npm install --no-audit --no-fund \
|| { err "npm install impossible."; exit 1; }
$SUDO npx prisma db push 2>/dev/null || npx prisma db push \
|| { err "prisma db push impossible."; exit 1; }
$SUDO npx prisma generate 2>/dev/null || npx prisma generate \
|| { err "prisma generate impossible."; exit 1; }
info "Compilation production (2 a 6 minutes)..."
$SUDO env NODE_OPTIONS="--max-old-space-size=2048" npm run build 2>/dev/null \
|| env NODE_OPTIONS="--max-old-space-size=2048" npm run build \
|| { err "Build impossible."; exit 1; }
[ -f .next/standalone/server.js ] \
|| { err "Build incomplet : .next/standalone/server.js absent."; exit 1; }
ok "Build production pret (.next/standalone)."
```

Partie 4 — Service systemd, pare-feu et vérification finale

Dernière ligne droite : un utilisateur système dédié signage est créé, les droits de /opt/signagecms lui sont transférés, puis l'unité signagecms.service est installée et activée (démarrage au boot, redémarrage automatique après crash, redémarrage toutes les 5 secondes si besoin). Le script ouvre ensuite le port dans ufw ou firewalld s'ils sont actifs, puis sonde l'API pendant 30 secondes maximum et affiche un récapitulatif complet (adresse du CMS, emplacement des identifiants admin, commandes de suivi des journaux).

```
# ---------- 7. service systemd ----------------------------------------
hr
if [ "$WITH_SERVICE" -eq 1 ] && [ "$SUDO_USABLE" -eq 0 ]; then
info "7/8 - Service systemd ignore : privileges root indisponibles."
warn "Relancez avec sudo pour installer le service, ou lancez a la main :"
warn " bash $APP_DIR/deploy/linux/start.sh"
elif [ "$WITH_SERVICE" -eq 1 ] && command -v systemctl >/dev/null 2>&1; then
info "7/8 - Service systemd (demarrage automatique)"
```

```
SVC_USER="signage"
if ! id "$SVC_USER" >/dev/null 2>&1; then
$SUDO useradd --system --create-home --shell /usr/sbin/nologin "$SVC_USER"
ok "Utilisateur systeme '$SVC_USER' cree."
fi
NODE_BIN="$($SUDO sh -c 'command -v node')"
$SUDO chown -R "$SVC_USER:$SVC_USER" "$APP_DIR"
$SUDO tee /etc/systemd/system/signagecms.service >/dev/null <<EOF
[Unit]
Description=SignageCMS - Serveur d'affichage dynamique
After=network.target

[Service]
Type=simple
User=$SVC_USER
WorkingDirectory=$APP_DIR
ExecStart=$NODE_BIN $APP_DIR/.next/standalone/server.js
Environment=NODE_ENV=production
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
$SUDO systemctl daemon-reload
$SUDO systemctl enable --now signagecms
ok "Service signagecms active (demarrage au boot + restart)."
else
info "7/8 - Service systemd ignore (--no-service ou systemctl absent)."
warn "Lancement manuel : bash $APP_DIR/deploy/linux/start.sh"
fi

# ---------- 8. pare-feu + verification ------------------------------
hr
if [ "$WITH_FIREWALL" -eq 1 ]; then
info "8/8 - Pare-feu : ouverture du port $CMS_PORT/tcp"
if command -v ufw >/dev/null 2>&1 \
&& $SUDO ufw status 2>/dev/null | grep -q "Status: active"; then
$SUDO ufw allow "$CMS_PORT/tcp" && ok "UFW : port $CMS_PORT ouvert."
elif command -v firewall-cmd >/dev/null 2>&1 \
&& $SUDO firewall-cmd --state >/dev/null 2>&1; then
$SUDO firewall-cmd --permanent --add-port="$CMS_PORT/tcp"
$SUDO firewall-cmd --reload
ok "firewalld : port $CMS_PORT ouvert."
else
warn "Aucun pare-feu actif detecte (ou firewalld/ufw absent) - rien a faire."
```

```
fi
else
info "8/8 - Pare-feu ignore (--no-firewall)."
fi

# ---------- verification finale ------------------------------------
hr
info "Verification finale : http://127.0.0.1:$CMS_PORT/api/version"
API_OK=0
for _ in $(seq 1 15); do
API_V="$(curl -s --max-time 2 "http://127.0.0.1:$CMS_PORT/api/version" \
2>/dev/null | tr -d '"')"
[ -n "$API_V" ] && { API_OK=1; break; }
sleep 2
done
hr
if [ "$API_OK" -eq 1 ]; then
ok "Serveur CMS v$API_V en ligne sur le port $CMS_PORT."
else
warn "Le serveur ne repond pas encore (build/service recents)."
if [ "$WITH_SERVICE" -eq 1 ]; then
warn "Consultez : journalctl -u signagecms -f"
else
warn "Demarrage : bash $APP_DIR/deploy/linux/start.sh"
fi
fi
echo " Adresse CMS : http://ADRESSE-IP-DU-SERVEUR:$CMS_PORT"
echo " Identifiants admin : $APP_DIR/db/admin-credentials.txt (1re connexion)"
echo " Journaux service : journalctl -u signagecms -f"
echo " Arret/relance : bash $APP_DIR/deploy/linux/start.sh --stop | --restart"
hr
```

Idempotence et mises à jour

install.sh peut être relancé pour mettre à jour un serveur existant : fournir la nouvelle archive (--zip) déclenche la ré-extraction, npm install, le build et le redémarrage du service, tout en conservant la base SQLite et le fichier .env. Pour un déploiement sans systemd (conteneur, superviseur maison), ajoutez --no-service et lancez le serveur avec le script start.sh décrit plus bas.

Déployer l'archive du serveur

Téléchargez l'archive serveur (lien en annexe A) puis extrayez-la dans /opt, emplacement conventionnel des applications locales. L'archive produit un dossier signagecms d'environ 1,4 Mo de sources ; les dépendances et le build occuperont ensuite plusieurs centaines de mégaoctets. Prévoyez également l'espace des médias diffusés, servis depuis le dossier public/uploads du projet.

```
sudo mkdir -p /opt && cd /opt
sudo unzip ~/Téléchargements/SignageCMS-Serveur.zip
sudo chown -R $USER:$USER /opt/signagecms
cd /opt/signagecms
```

Outils de conversion PowerPoint / PDF (optionnel)

Le script livré dans le projet détecte automatiquement votre distribution (apt, dnf, yum, zypper, pacman, apk), installe LibreOffice et Poppler depuis les paquets officiels puis vérifie le résultat. Il est idempotent : le relancer ne réinstalle rien. Sans sudo, l'option --no-sudo télécharge l'archive officielle de LibreOffice dans votre dossier personnel ; déclarez alors le chemin dans le fichier .env via SOFFICE_PATH (détails dans deploy/linux/LISEZMOI.txt).

```
sudo bash deploy/linux/install-tools.sh
soffice --version && pdftoppm -v # vérification
```

Configurer le fichier .env

Créez un fichier **.env** à la racine du projet. La seule variable indispensable est DATABASE_URL, qui doit pointer vers un chemin **absolu** avec des slashes « / », même sous Windows. Le dossier db est créé automatiquement à la préparation de la base.

```
cat > .env <<'EOF'
DATABASE_URL=file:/opt/signagecms/db/custom.db
# PORT=3000
# MAX_UPLOAD_MB=500
# SOFFICE_PATH=/chemin/vers/soffice
# PDFTOPPM_PATH=/chemin/vers/pdftoppm
EOF
```

Dépendances, base de données et compilation

Installez les dépendances npm (ou bun install si vous utilisez Bun), préparez le schéma Prisma puis compilez la version production. Le build produit un dossier **.next/standalone** autonome : c'est lui que le serveur exécute en production, avec un simple Node.js, sans dépendance au dossier node_modules complet.

```
npm install
npx prisma db push && npx prisma generate
npm run build
```

Premier test manuel

Avant d'installer le service, lancez le serveur au premier plan pour valider l'installation. Au premier démarrage, le compte admin est créé et son mot de passe aléatoire s'affiche dans la console (et dans db/admin-credentials.txt). Interrompez ensuite avec Ctrl+C avant de passer au service systemd.

```
npm run start:standalone
# ou, de façon équivalente, avec le script livré (voir section suivante) :
bash deploy/linux/start.sh
# dans une autre console :
curl -s http://localhost:3000/api/version
```

Lancer le serveur — le script start.sh

Le fichier **deploy/linux/start.sh**, également livré dans l'archive serveur et téléchargeable séparément, lance le serveur en production avec les mêmes contrôles qu'un exploitant vérifierait à la main : présence de Node.js, existence du fichier .env (créé minimalement s'il manque), build production présent, port lu dans .env (défaut 3000). Il propose quatre modes : au premier plan pour l'observation directe, en arrière-plan avec journal et fichier PID, plus les commandes d'état, d'arrêt et de redémarrage. C'est le moyen le plus simple de faire tourner le CMS quand le service systemd n'a pas été installé (option --no-service, conteneur, poste de test).

Commande	Effet
bash start.sh	Lancement au premier plan (Ctrl+C pour arrêter).
bash start.sh --daemon	Lancement en arrière-plan : journal service.log, PID dans server.pid.
bash start.sh --status	État du processus et version renvoyée par l'API.
bash start.sh --restart	Arrêt puis relance de l'instance en arrière-plan.
bash start.sh --stop	Arrêt de l'instance en arrière-plan.

Le script est reproduit ci-dessous en trois parties. Après un démarrage réussi, il affiche l'adresse de l'interface, l'emplacement des identifiants admin du premier lancement et la commande de suivi du journal ; en cas d'échec, il imprime les vingt dernières lignes de service.log pour un diagnostic immédiat.

Partie 1 — En-tête et modes d'emploi

```
#!/usr/bin/env bash
# ============================================================
# SignageCMS — Lancement du serveur (build production standalone)
```

```
# ------------------------------------------------------------
# Usage :
# bash start.sh # premier plan (Ctrl+C pour arreter)
# bash start.sh --daemon # arriere-plan (service.log + server.pid)
# bash start.sh --stop # arrete l'instance en arriere-plan
# bash start.sh --restart # redemarre l'instance en arriere-plan
# bash start.sh --status # etat du processus + version API
#
# Le script verifie Node.js, le fichier .env et le build
# production, puis demarre .next/standalone/server.js.
# Au premier demarrage, le compte admin est cree et son mot de
# passe aleatoire est ecrit dans db/admin-credentials.txt.
# ============================================================
set -u

APP_DIR="$(cd "$(dirname "$0")/../.." && pwd)"
cd "$APP_DIR" || { echo "Dossier introuvable : $APP_DIR"; exit 1; }

MODE="${1:-}"

ok() { printf '\033[32m[OK]\033[0m %s\n' "$*"; }
warn() { printf '\033[33m[!]\033[0m %s\n' "$*"; }
err() { printf '\033[31m[ERREUR]\033[0m %s\n' "$*"; }

PID_FILE="$APP_DIR/server.pid"
LOG_FILE="$APP_DIR/service.log"
```

Partie 2 — Contrôles d'environnement avant lancement

Avant tout démarrage, le script valide la présence de Node.js, crée un .env minimal si nécessaire, recharge les variables DATABASE_URL et PORT depuis ce fichier, vérifie que le build production existe et prépare les fonctions de contrôle du processus et de l'API. Tout écart est signalé avec la commande exacte à exécuter pour y remédier.

```
# ---------- verification de l'environnement --------------------------
if ! command -v node >/dev/null 2>&1; then
err "Node.js absent. Installez-le d'abord (voir deploy/linux/install.sh)."
exit 1
fi

if [ ! -f .env ]; then
warn "Fichier .env absent - creation minimale (DATABASE_URL local)."
mkdir -p db
echo "DATABASE_URL=file:$APP_DIR/db/custom.db" > .env
fi
```

```
# charge DATABASE_URL et PORT definis dans .env (sans ecraser l'env existant)
set -a; . ./.env; set +a

if [ ! -f .next/standalone/server.js ]; then
err "Build production absent (.next/standalone/server.js)."
echo " Lancez : npm install && npx prisma db push && npm run build"
echo " Ou directement : sudo bash deploy/linux/install.sh"
exit 1
fi

# port : .env puis variable PORT puis 3000
PORT="${PORT:-3000}"

is_running() {
[ -f "$PID_FILE" ] && kill -0 "$(cat "$PID_FILE" 2>/dev/null)" 2>/dev/null
}

api_version() {
curl -s --max-time 2 "http://127.0.0.1:$PORT/api/version" 2>/dev/null | tr -d '"'
}
```

Partie 3 — Modes de lancement, arrêt, état et redémarrage

```
# ---------- commandes ------------------------------------------------
case "$MODE" in
--stop)
if is_running; then
PID="$(cat "$PID_FILE")"
kill "$PID" && rm -f "$PID_FILE"
ok "Serveur arrete (pid $PID)."
else
warn "Aucune instance en arriere-plan (server.pid absent)."
fi
exit 0 ;;

--status)
if is_running; then
ok "Processus actif : pid $(cat "$PID_FILE") - port $PORT"
else
warn "Aucune instance en arriere-plan."
fi
V="$(api_version)"
if [ -n "$V" ]; then
ok "API repond : version $V sur http://127.0.0.1:$PORT"
else
```

```
warn "API sans reponse sur le port $PORT."
fi
exit 0 ;;

--restart)
"$0" --stop
sleep 2
exec "$0" --daemon ;;

--daemon)
if is_running; then
warn "Deja actif : pid $(cat "$PID_FILE") - utilisez --restart pour relancer."
exit 0
fi
echo "Demarrage en arriere-plan : service.log + server.pid"
nohup env NODE_ENV=production PORT="$PORT" \
node .next/standalone/server.js >> "$LOG_FILE" 2>&1 &
echo $! > "$PID_FILE"
sleep 3
if is_running; then
ok "Serveur demarre : pid $(cat "$PID_FILE") - port $PORT"
V="$(api_version)"
[ -n "$V" ] && ok "Version API : $V"
echo " Interface CMS : http://ADRESSE-IP-DU-SERVEUR:$PORT"
echo " Admin (1er lancement) : $APP_DIR/db/admin-credentials.txt"
echo " Journaux : tail -f $LOG_FILE"
echo " Arret : bash $0 --stop"
else
err "Echec du demarrage - consultez $LOG_FILE"
tail -n 20 "$LOG_FILE" 2>/dev/null
exit 1
fi
exit 0 ;;

*)
echo "SignageCMS - lancement au premier plan (Ctrl+C pour arreter)"
echo " Port : $PORT Journal : console"
echo
exec env NODE_ENV=production PORT="$PORT" node .next/standalone/server.js ;;
esac
```

> **start.sh ou service systemd ?**
> Si l'unité signagecms.service a été créée par install.sh, préférez systemctl (démarrage au boot, redémarrage après crash, journaux journald) : sudo systemctl restart signagecms. Le script start.sh reste utile pour les tests, les conteneurs et les environnements sans systemd — les deux approches utilisent exactement le même build .next/standalone.

Service systemd (démarrage automatique)

L'étape 7/8 de install.sh crée automatiquement cette unité — la procédure ci-dessous ne concerne donc que l'installation manuelle. Elle rend le serveur démarrable au boot, redémarré en cas de crash et journalisé dans journald. Adaptez l'utilisateur (ici signage, dédié au service) et le chemin d'installation si nécessaire, puis activez le service.

```
sudo tee /etc/systemd/system/signagecms.service <<'EOF'
[Unit]
Description=SignageCMS - Serveur d'affichage dynamique
After=network.target

[Service]
Type=simple
User=signage
WorkingDirectory=/opt/signagecms
ExecStart=/usr/bin/node /opt/signagecms/.next/standalone/server.js
Environment=NODE_ENV=production
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now signagecms
systemctl status signagecms --no-pager
journalctl -u signagecms -f # suivre les journaux en direct
```

Pare-feu et accès réseau

Ouvrez le port 3000 dans le pare-feu local pour que les écrans rejoignent le serveur. Sur Ubuntu avec UFW : sudo ufw allow 3000/tcp. Vérifiez l'accès depuis un autre poste avec http://ADRESSE-IP-DU-SERVEUR:3000. L'adresse IP du serveur s'obtient avec ip addr ou hostname -I ; le CMS l'affiche également dans le dialogue « Connecter Android » et dans les Paramètres, rubrique informations système.

Reverse proxy HTTPS (recommandé hors réseau local)

Dès que le CMS quitte le réseau local (multi-sites, accès par Internet), placez-le derrière un reverse proxy TLS. Caddy est le plus simple : deux lignes de configuration suffisent et le certificat Let's Encrypt est géré automatiquement. Le fichier Caddyfile fourni à la racine du projet sert de point de départ ; lancez ensuite caddy run.

```
cms.exemple.fr {
reverse_proxy localhost:3000
}
```

5. Premier démarrage et première connexion

Une fois le serveur démarré (start-cms.bat sous Windows, service systemd sous Linux), l'interface d'administration est accessible depuis le serveur lui-même à l'adresse **http://localhost:3000**, ou depuis n'importe quel poste du réseau à **http://ADRESSE-IP-DU-SERVEUR:3000**. La page de connexion présente un formulaire à deux champs : identifiant et mot de passe. Les écrans, eux, ne passent jamais par cette page : les lecteurs se connectent directement avec leur clé matérielle (chapitre 7).

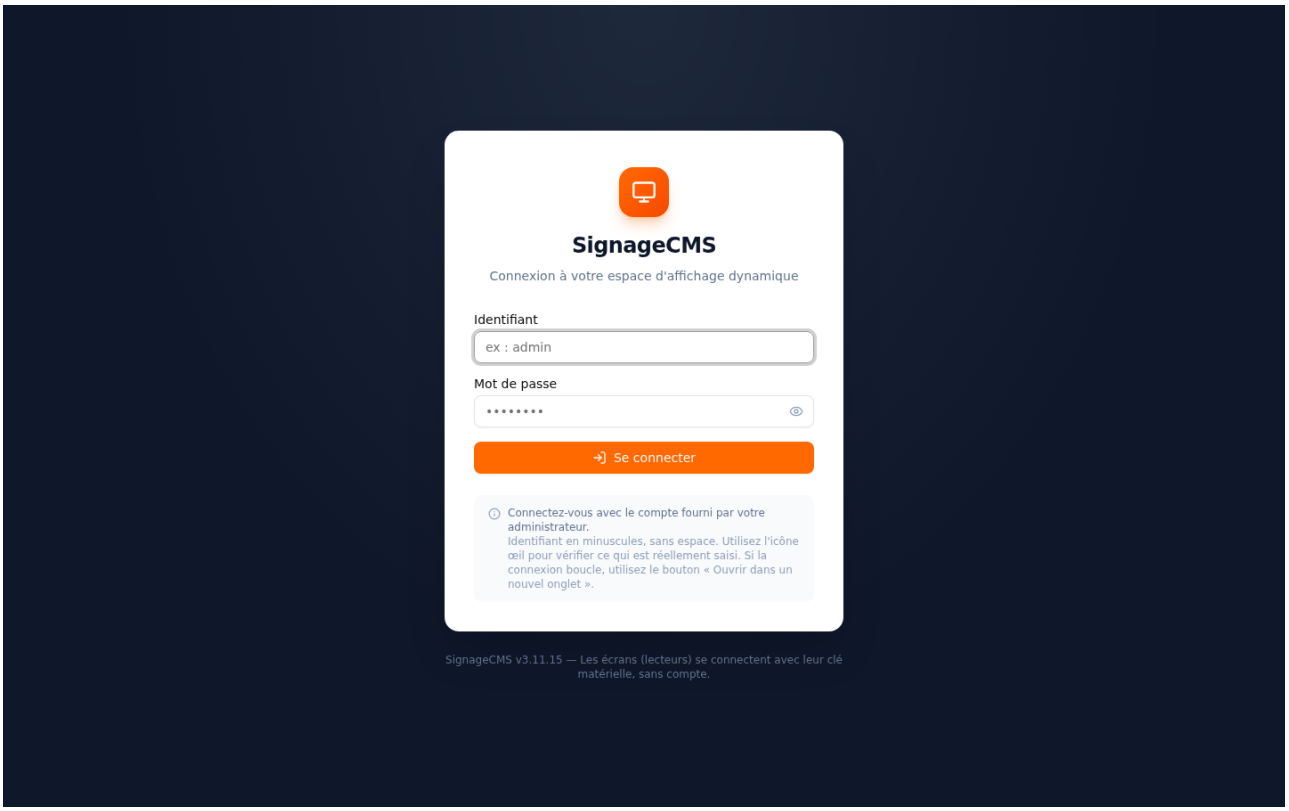


Figure 2 — Page de connexion de l'interface d'administration

Compte administrateur initial

1. Saisissez l'identifiant **admin**.
2. Récupérez le **mot de passe initial** : il n'existe aucun mot de passe par défaut — le serveur en génère un aléatoire à la première création du compte. Il s'affiche une fois dans la console du serveur et est enregistré dans **db/admin-credentials.txt**.
3. Cliquez sur « Se connecter » : le tableau de bord s'ouvre.
4. Changez immédiatement le mot de passe via le menu **Utilisateurs**, puis **supprimez** le fichier db/admin-credentials.txt.

> **Sécurité — à faire dès la première connexion**
>
> Le mot de passe initial étant écrit dans un fichier texte, changez-le dès la première connexion puis supprimez db/admin-credentials.txt. Créez ensuite des comptes nominatifs avec le rôle le plus restrictif possible (user ou read-only) plutôt que de partager le compte admin. Chaque réinitialisation de mot de passe révoque immédiatement toutes les sessions ouvertes du compte.

Rôles disponibles

Trois rôles standard structurent les droits : **admin** dispose de toutes les fonctions ; **user** accède à tout sauf à la gestion des comptes et des paramètres en écriture ; **read-only** consulte sans modifier. La matrice précise des actions (voir, créer, modifier, supprimer) est réglable module par module depuis « Rôles et droits » et est appliquée côté serveur sur chaque appel d'API. SignageCMS peut également déléguer l'authentification à un annuaire LDAP ou Active Directory (Paramètres, section LDAP) : les comptes sont alors importés d'une unité d'organisation choisie, avec rôle et groupe par défaut, sans jamais toucher au compte admin local.

Mot de passe administrateur perdu

Si aucun autre compte admin n'est disponible, la procédure de secours consiste à supprimer le compte admin de la base (ou la base elle-même sur une installation neuve) puis à relancer le serveur : un nouveau mot de passe aléatoire est généré et affiché dans la console ainsi que dans db/admin-credentials.txt. Sur une installation Windows, arrêtez d'abord le service (nssm stop SignageCMS), effectuez la manipulation, puis redémarrez. Un administrateur connecté peut aussi réinitialiser le mot de passe de n'importe quel compte depuis le menu Utilisateurs, ce qui déconnecte immédiatement ses sessions en cours.

6. Prise en main de l'interface

Structure générale

L'interface s'organise autour d'une barre latérale sombre divisée en deux groupes. Le groupe **Général** rassemble les modules opérationnels : Tableau de bord, Écrans, Bibliothèque de médias, TV / IPTV, Présentations, Playlists, Campagnes et Planification. Le groupe **Administration** réunit Rapports, Utilisateurs, Paramètres et Rôles et droits. La barre supérieure donne accès à la recherche globale, aux notifications, au sélecteur de compte et à la déconnexion. La version du serveur est rappelée en bas de la barre latérale.

Module	Rôle
Tableau de bord	Vue d'ensemble temps réel : statistiques, activité 7 jours, journal d'activité, état du parc.
Écrans	Inventaire des lecteurs, clés matérielles, groupes, aperçu à distance, connexion Android.
Bibliothèque de médias	Import et gestion de tous les contenus diffusables (images, vidéos, documents).
TV / IPTV	Sources Xtream et listes M3U, chaînes, proxy et partage des connexions.
Présentations	Mises en page par régions et widgets (concepteur visuel intégré).
Playlists	Séquences ordonnées de médias avec durée par élément.
Campagnes	Enchaînement de présentations en séquence thématique.
Planification	Calendrier des événements : quoi, quand, où, boucle ON/OFF.
Rapports	Journaux serveur et preuves de diffusion (proof-of-play) exportables.
Utilisateurs	Comptes locaux, rôles, réinitialisations de mot de passe, import LDAP.
Paramètres	Identité du CMS, réglages lecteur, taille d'upload, informations système, maintenance.
Rôles et droits	Matrice de permissions fine par module et par rôle.

Le tableau de bord

Le tableau de bord est la page d'accueil après connexion. Les cartes de statistiques comptent les écrans connectés (ici 1 sur 1 en ligne), les médias, les présentations, les chaînes IPTV et les planifications, avec leur évolution récente ; un clic sur certaines cartes ouvre directement le module concerné. Le graphique « Consommation du réseau » retrace les 7 derniers jours à partir des preuves de diffusion, tandis que la jauge « État du parc d'écrans » distingue les écrans en ligne et hors ligne. En bas de page, le journal d'activité liste les derniers événements serveur — idéal pour vérifier qu'un écran vient bien de se connecter — et l'aperçu rapide donne accès aux éléments récents.

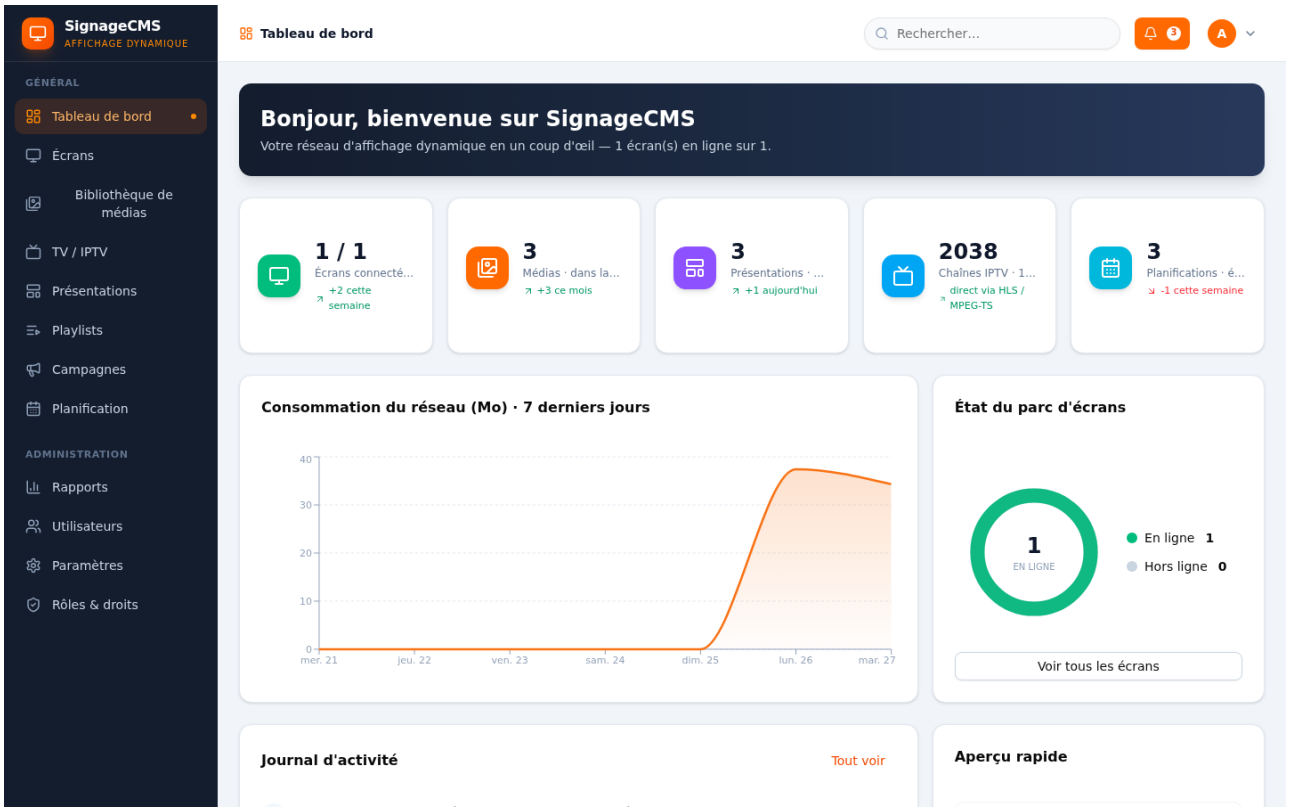


Figure 3 — Tableau de bord : statistiques, consommation réseau, état du parc

Les Paramètres

Le module Paramètres se compose de quatre onglets. **Général** définit l'identité du CMS (nom affiché, e-mail de l'administrateur), les statistiques d'utilisation et les notifications e-mail en cas d'écran hors ligne. **Affichage** regroupe les réglages des lecteurs (intervalles de synchronisation et de battements de cœur, preuves de diffusion) et la taille maximale des téléversements. **Régional** couvre langue et formats. **Maintenance** enfin expose les informations système utiles au diagnostic : version du CMS, nom d'hôte et adresses IP réseau réelles du serveur, utilisées notamment par le dialogue « Connecter Android ».

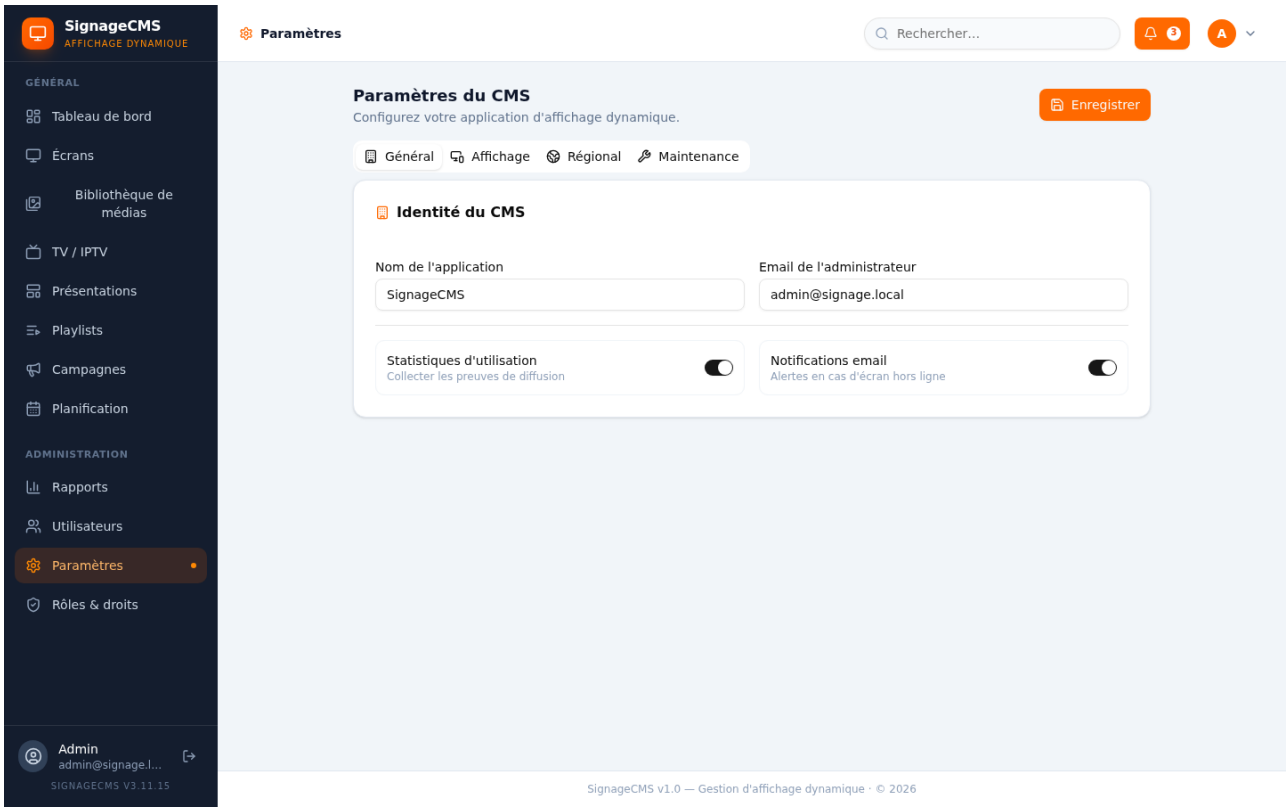


Figure 4 — Paramètres du CMS : onglets Général, Affichage, Régional, Maintenance

7. Écrans : création et connexion des lecteurs

Le principe de la clé matérielle

Le module **Écrans** gère l'inventaire des lecteurs. Chaque écran possède un nom, une **clé matérielle** unique (par exemple KEY-MVP336RJ), une résolution, un statut en ligne / hors ligne maintenu par des battements de cœur réguliers et une date de dernier accès. C'est cette clé — et non un compte utilisateur — qui relie l'écran physique au CMS : elle est saisie (ou scannée) une seule fois sur le lecteur, puis tous les contenus planifiés pour cet écran arrivent automatiquement.

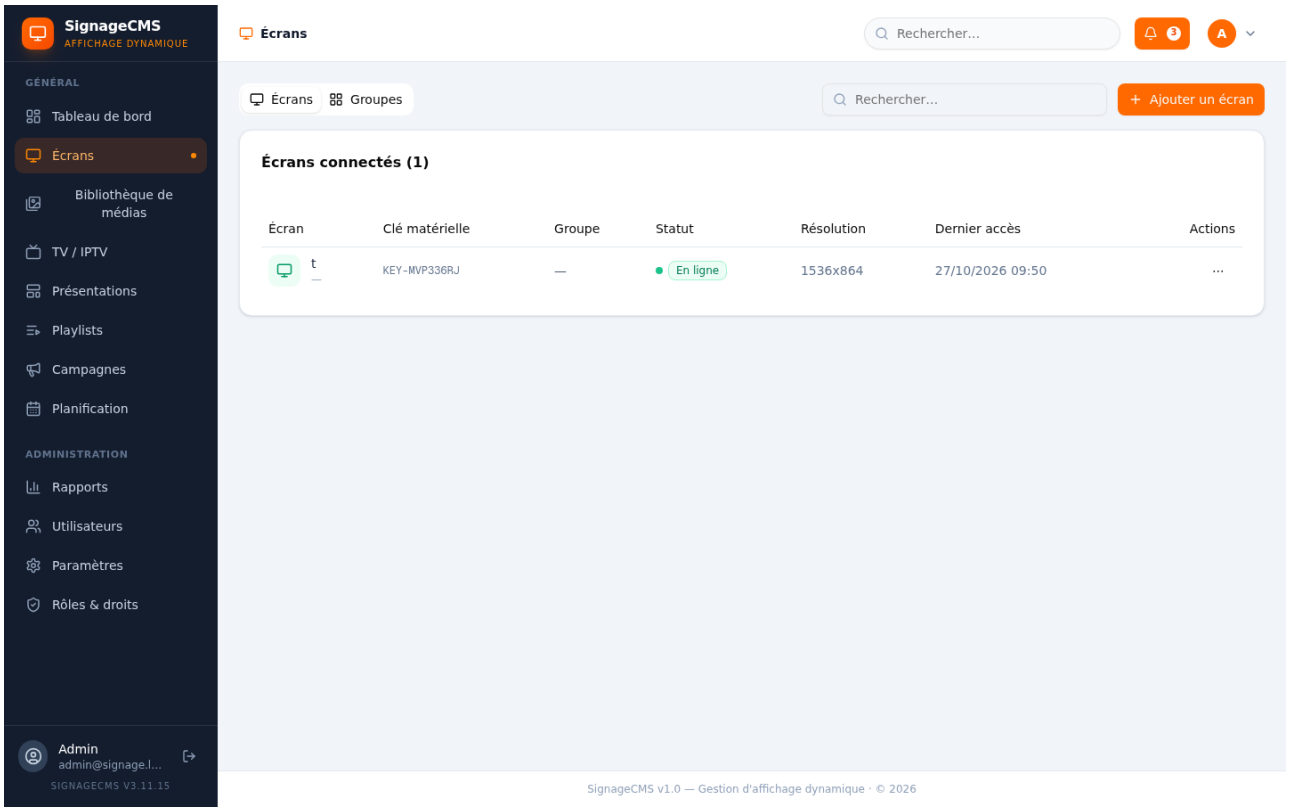


Figure 5 — Liste des écrans connectés avec clé matérielle, statut et résolution

Créer un écran dans le CMS

1. Ouvrez **Écrans** puis cliquez sur **+ Ajouter un écran**.
2. Saisissez un nom explicite lié à l'emplacement physique (par exemple « Hall d'accueil »), facultativement une description, un groupe et une résolution.
3. Enregistrez : l'écran apparaît dans la liste avec sa clé matérielle, d'abord hors ligne.

Connecter le lecteur web

1. Sur l'écran, ouvrez un navigateur (Chrome, Edge, Firefox) à l'adresse **http://SERVEUR:3000/player**.
2. Cliquez sur la roue dentée du lecteur pour ouvrir les réglages.
3. Choisissez votre écran dans la liste proposée, ou saisissez directement sa clé matérielle — la liaison est immédiate.
4. Passez le navigateur en plein écran (touche F11) ou utilisez le mode kiosk (chapitre 11) pour un affichage sans barres.

Connecter l'application Android

1. Installez l'APK SignageCMS Player sur l'appareil (téléviseur Android, tablette, box) — voir le chapitre 11 pour les versions disponibles.

2. Dans le CMS, ouvrez **Écrans**, puis le menu « ... » de l'écran et choisissez **Connecter Android**.
3. Le dialogue affiche l'adresse réseau correcte du serveur (localhost y est automatiquement remplacé), la clé de l'écran et un **QR code**.
4. Saisissez ces informations dans l'application ou scannez le QR code, puis utilisez le bouton **Tester** pour vérifier que l'APK répond bien au serveur.

Groupes, affectation et surveillance

Les **groupes d'écrans** (onglet Groupes de la page Écrans) permettent de cibler une planification entière en une fois : créez par exemple « Vitrines », « Étages » ou « Sites ». Un écran n'appartient qu'à un groupe à la fois et peut être déplacé à tout moment. La **présentation par défaut** affectée à l'écran ou au groupe s'affiche quand aucun événement planifié n'est actif — c'est le contenu de fond de votre écran. Le menu « ... » donne aussi accès à l'**aperçu en direct** (visualisez ce que montre réellement l'écran) et au mode incognito, qui masque le nom de l'écran dans les listes. Un écran vu en ligne il y a plus de quelques minutes repasse automatiquement hors ligne : aucune manipulation manuelle n'est nécessaire.

> **Bonne pratique**
>
> Nommez les écrans selon leur emplacement physique et regroupez-les par zone de diffusion dès la mise en service. Cette discipline rend la planification beaucoup plus lisible : un événement ciblé sur le groupe « Vitrines » s'applique automatiquement à tout nouvel écran ajouté au groupe.

8. Médias, playlists et présentations

La bibliothèque de médias

Le module **Bibliothèque de médias** centralise tout ce qui peut être diffusé. L'import se fait par glisser-déposer ou par le bouton d'import, avec une barre de progression et des messages d'erreur explicites en cas de refus (format non supporté, taille dépassée, espace disque insuffisant — le serveur refuse l'import si l'espace libre passe sous 100 Mo). Chaque média possède une fiche détaillée : type, taille, étiquettes et nombre de diapositives pour les documents convertis. La modification permet de renommer le média et de lui attribuer des étiquettes. Le champ « Durée (s) » n'apparaît pas dans les dialogues d'import et de modification : c'est une métadonnée sans effet sur la lecture — la durée d'affichage réelle se règle directement sur chaque widget de présentation ou élément de playlist, et c'est elle que les lecteurs web et Android appliquent.

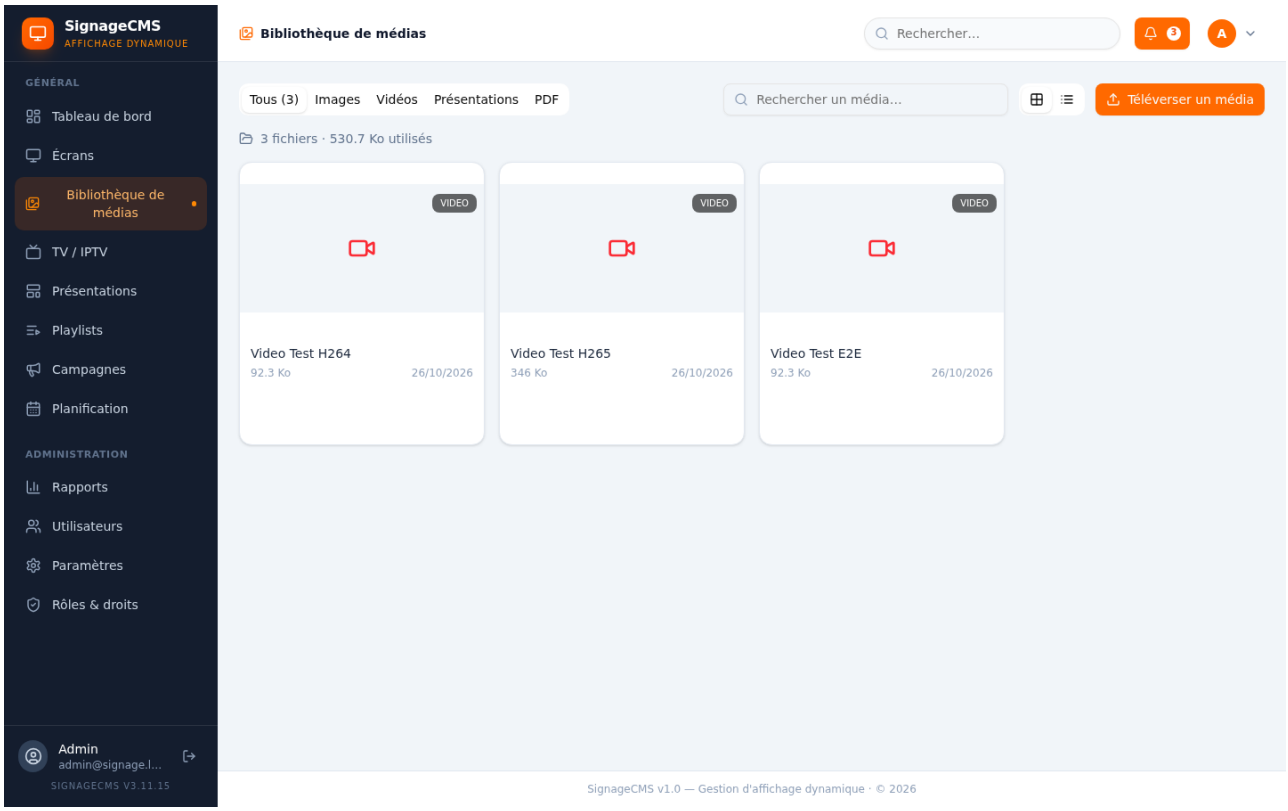


Figure 6 — Bibliothèque de médias : import, recherche et fiches

Catégorie	Formats acceptés
Images	PNG, JPG, JPEG, GIF, WebP, SVG, BMP, ICO, TIFF
Vidéos	MP4, WebM, MOV, MKV, AVI, M4V, MPEG/MPG, 3GP, TS
Audio	MP3, WAV, OGG, M4A, AAC, FLAC
Documents	PDF
Présentations	PPT, PPTX, PPS, PPSX, ODP
Données	JSON, CSV, XLSX, XLS (widgets de données)

Conversion automatique des PowerPoint et PDF

Un fichier PowerPoint ou PDF importé est automatiquement converti en suite de **diapositives images**, exploitables comme n'importe quel média dans les playlists, les régions de présentation et les campagnes. La conversion s'appuie sur LibreOffice (PPT vers PDF) et Poppler (PDF vers PNG) installés au chapitre 3 ou 4. Sans ces outils, le fichier est accepté mais reste marqué « Non converti » ; dès que les outils deviennent disponibles, la conversion repart automatiquement (environ 60 secondes) — un bouton **Reconvertir** est également disponible sur chaque média concerné.

Playlists

Une **playlist** est une séquence ordonnée de médias joués l'un après l'autre. Dans le module Playlists, créez la liste, ajoutez des médias depuis la bibliothèque, réordonnez par glisser-déposer et réglez la durée d'affichage de chaque élément : les images et documents suivent cette durée, tandis que les vidéos et audios se jouent en entier par défaut. Les playlists servent principalement de contenu d'une **région** de présentation : une colonne d'annonces qui tourne en boucle, une bande d'images, une zone de textes défilants alimentée par un fichier CSV.

Présentations (layouts) et concepteur

La **présentation** définit la mise en page d'un écran : un fond et une ou plusieurs **régions** rectangulaires, chacune remplie par un ou plusieurs widgets. Le concepteur intégré permet de tout faire à la souris : créez la présentation avec la résolution cible (1920 x 1080 par défaut), positionnez et redimensionnez les régions par glisser-déposer ou en saisissant des coordonnées précises, puis affectez un widget à chaque région. L'aperçu montre le rendu exact avant enregistrement, et chaque région est zoomable.

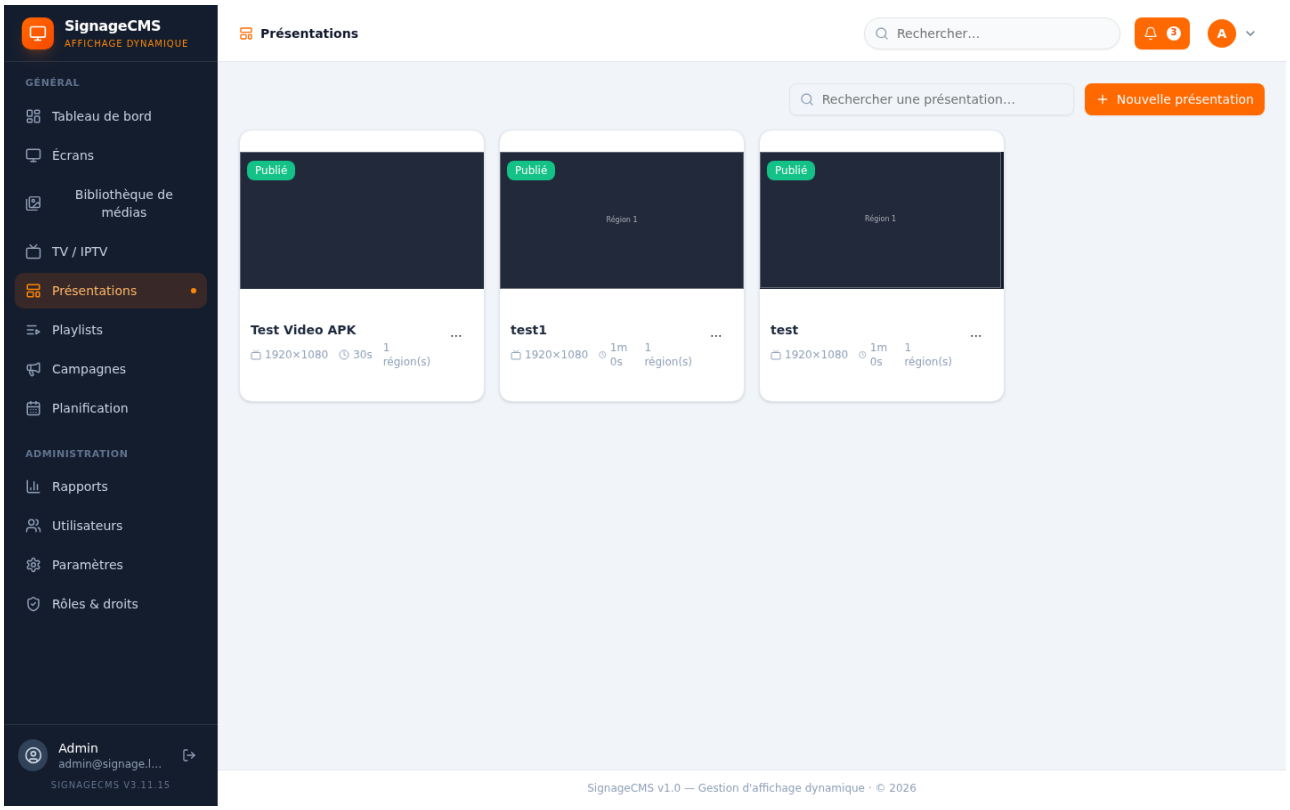


Figure 7 — Module Présentations : liste des mises en page créées

Widget	Rôle
Média	Image, vidéo, audio ou diapositive de la bibliothèque
Playlist	Séquence de médias en rotation dans une région
Texte	Titre ou bloc de texte libre (taille, couleur)

Widget	Rôle
Texte défilant	Bandeau horizontal continu (marquee)
Horloge	Heure au format 24 h, habillage réglable
Météo	Conditions et température d'une ville
Page web	Site ou application web intégrée, avec proxy anti-iframe
IPTV	Flux TV diffusé dans une région via le proxy intégré
Compte à rebours	Jours et heures restantes avant une date clé
RSS	Fil d'actualités avec titres défilants

La même présentation peut être affectée à plusieurs écrans ou groupes : les lecteurs la récupèrent automatiquement et l'adaptent à leur résolution réelle. Une présentation bien conçue — par exemple un bandeau d'annonces en haut, une zone vidéo au centre et une horloge en bas — devient un gabarit réutilisable pour tout le parc.

9. Campagnes

Une **campagne** regroupe plusieurs **présentations jouées en séquence** : idéal pour les programmations thématiques du type matinée / midi / soir ou pour des rotations commerciales hebdomadaires. À la diffusion, l'écran enchaîne toutes les présentations de la campagne dans l'ordre défini, chacune pendant la durée qui lui est attribuée. La campagne se comporte ensuite comme une présentation unique dans la planification : elle peut être ciblée par un événement ou affectée comme présentation par défaut d'un écran.

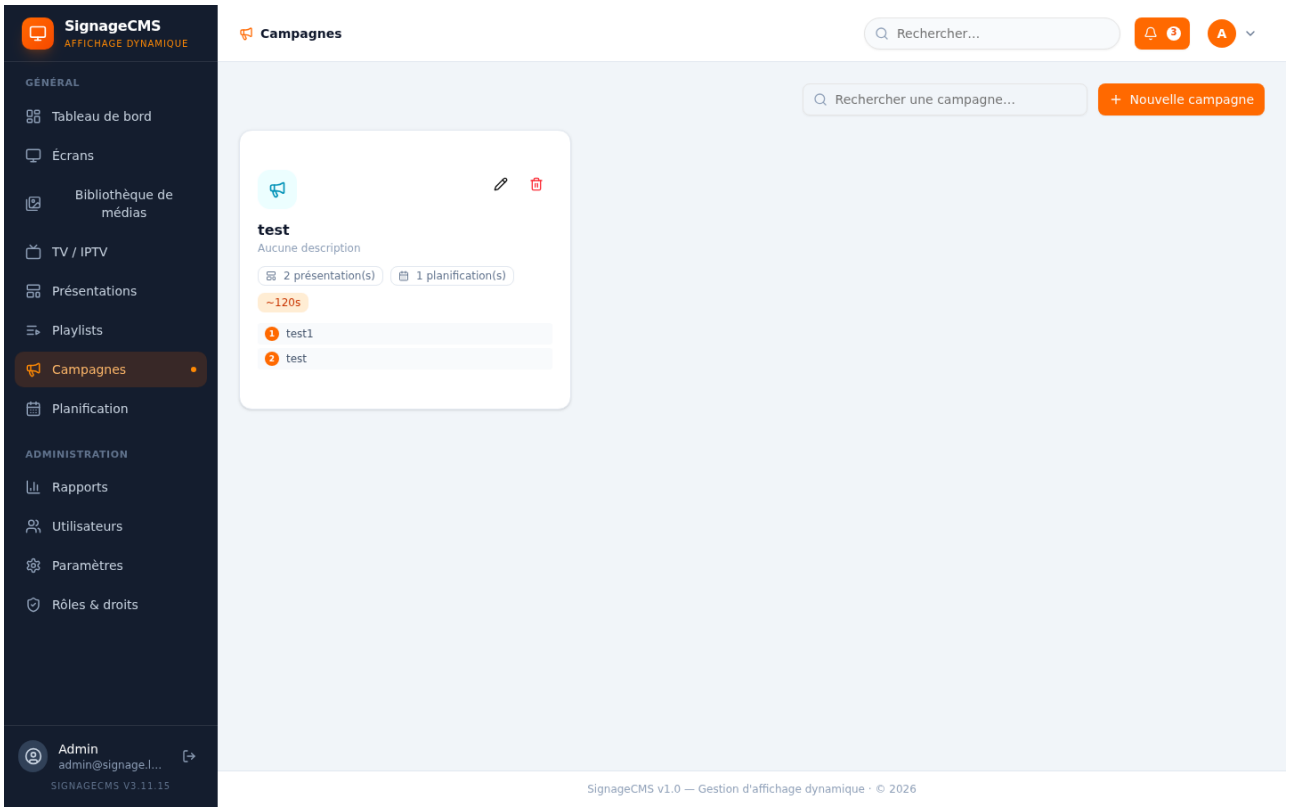


Figure 8 — Module Campagnes : liste et création

Créer une campagne

1. Ouvrez **Campagnes** puis cliquez sur **Nouvelle campagne** ; donnez-lui un nom explicite (par exemple « Vitrine — programme du jour »).
2. Cliquez sur **Gérer les présentations** pour ouvrir le dialogue de composition.
3. Ajoutez les présentations dans l'ordre voulu avec le sélecteur « Choisir une présentation », puis réglez la **durée** de chacune.
4. Réordonnez les éléments avec les boutons Monter / Descendre, puis fermez le dialogue : la séquence est enregistrée.

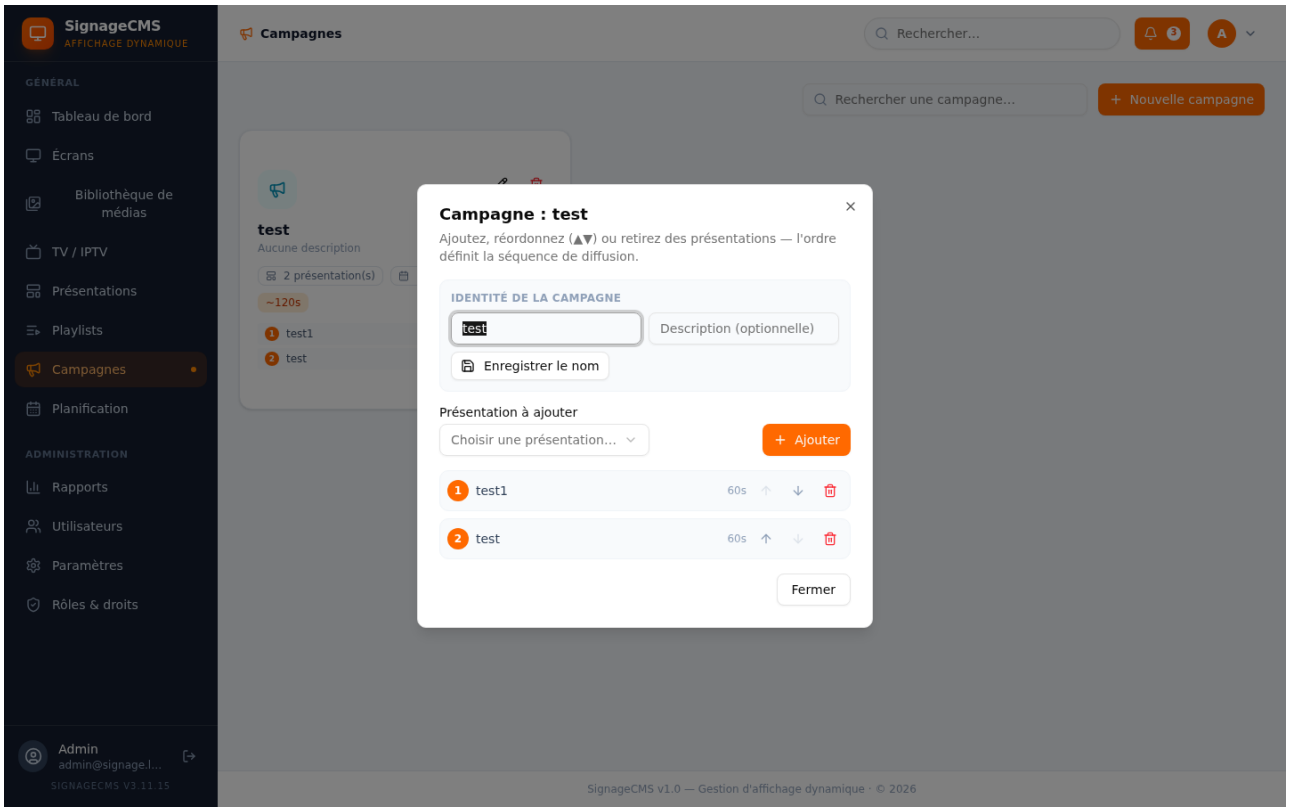


Figure 9 — Composition d'une campagne : ordre et durée des présentations

Comportement de la boucle à la fin de la séquence

La boucle de l'événement de planification s'applique aussi aux campagnes. Avec la boucle **ON**, la séquence reprend immédiatement au début, en cycle infini. Avec la boucle **OFF**, la campagne est jouée **une seule fois** — chaque présentation respecte exactement sa durée — puis le lecteur quitte le contenu : il passe à l'événement suivant encore actif, sinon il affiche l'écran d'attente (horloge) jusqu'à la fin de la fenêtre de diffusion. Pour rejouer une campagne en boucle OFF déjà diffusée, modifiez la fenêtre horaire de l'événement ou réactivez la boucle : l'événement est alors considéré comme neuf.

> **À retenir**
>
> Boucle OFF = la campagne est diffusée une fois, durées respectées, puis l'écran sort du contenu (événement suivant ou écran d'attente horloge). Plus de figeage sur la dernière présentation. La relecture exige de modifier la fenêtre de l'événement ou de réactiver la boucle.

10. Planification de la diffusion

Le calendrier hebdomadaire

Le module **Planification** est le cœur opérationnel du système : il programme quoi afficher, quand et où. La vue principale est un calendrier hebdomadaire — chaque colonne représente un jour, chaque événement occupe sa plage horaire et les événements chevauchants sont répartis en colonnes lisibles plutôt que superposés. La navigation par semaine se fait avec les flèches, et le compteur d'événements planifiés est affiché en haut de page.

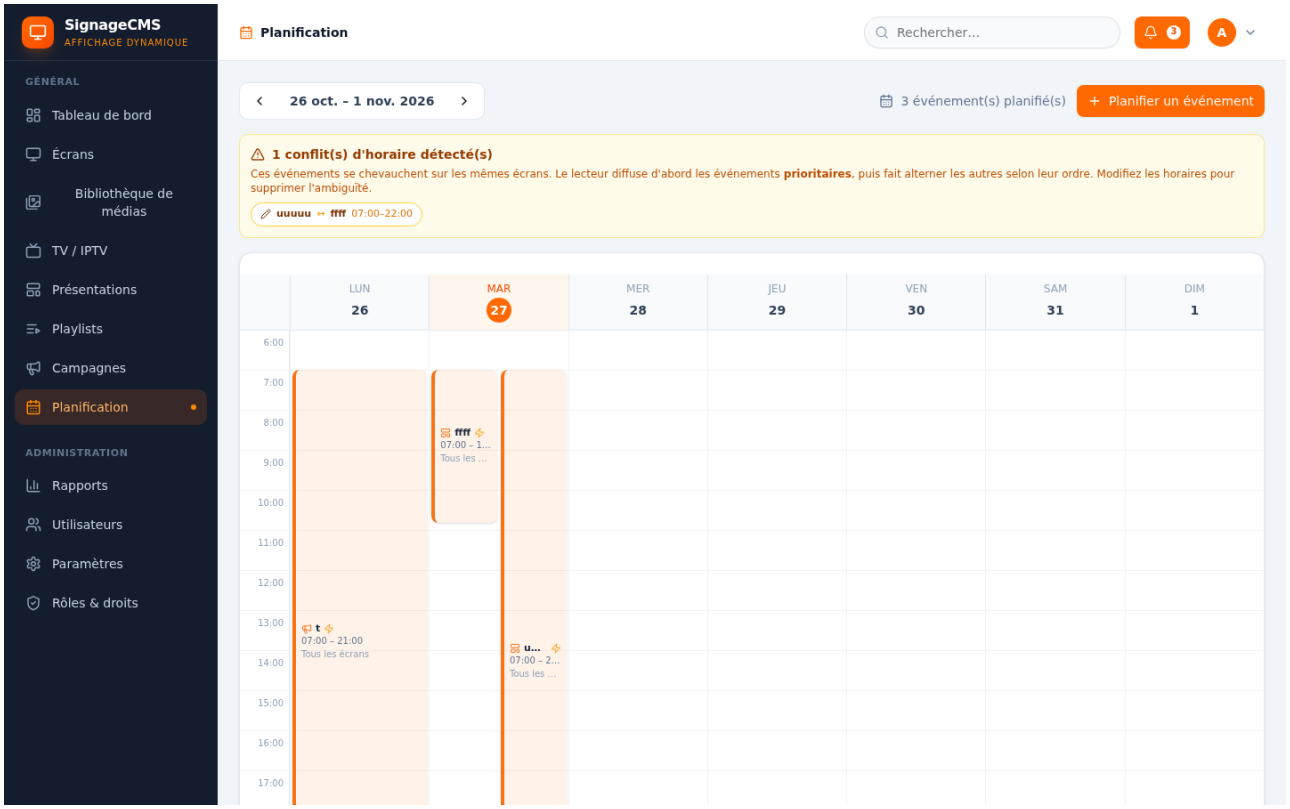


Figure 10 — Calendrier de planification : événement quotidien 07:00 - 21:00

Créer un événement

1. Cliquez sur **+ Planifier un événement**.
2. Donnez un titre, puis choisissez le contenu : une **présentation** ou une **campagne**.
3. Choisissez la cible : **tous les écrans**, un **groupe** d'écrans, ou une **liste** d'écrans en sélection multiple.
4. Définissez la **fenêtre de diffusion** : date et heure de début, date et heure de fin (format 24 h).
5. Réglez le commutateur **Répéter en boucle** (ON par défaut) et, si besoin, **Priorité haute** (voir plus bas), puis enregistrez.

Après enregistrement, l'événement apparaît dans le calendrier et les écrans ciblés le prennent en compte à leur prochaine synchronisation — 60 secondes maximum, sans intervention sur l'écran. Tous les champs restent modifiables par la suite (contenu, cibles, horaires, boucle) : les corrections sont propagées aux lecteurs sans recréer l'événement, et la suppression d'un événement fait reprendre aux écrans leur programmation résiduelle (autres événements actifs ou présentation par défaut).

Boucle ON / OFF par événement

Le réglage de boucle est **persisté en base, événement par événement** : il survit aux redémarrages du serveur et des lecteurs et se modifie à tout moment. En **ON**, le contenu reprend au début dès qu'il atteint sa fin (comportement boucle infinie). En **OFF**, le contenu est joué une seule fois — pour une campagne, chaque présentation respecte sa durée — puis le lecteur quitte le contenu vers l'événement suivant ou l'écran d'attente. La liste « Prochaines diffusions » et les info-bulles du calendrier affichent le badge Boucle ON/OFF de chaque événement, ce qui permet de vérifier d'un coup d'oeil le comportement programmé.

Conflits, priorité haute et ordre de passage

Si plusieurs événements ciblent le même écran sur la même plage horaire, ils sont tous diffusés en rotation et le serveur fixe l'ordre de passage : les événements marqués **Priorité haute** (éclair jaune dans le calendrier et la liste) passent en tête, devant les événements normaux ; entre deux événements de même niveau, l'ordre de création décide. Un événement prioritaire ne supprime pas les autres : dès l'ouverture de sa fenêtre, il reprend la tête de la rotation à la prochaine synchronisation du lecteur (60 secondes maximum), puis les autres contenus actifs enchaînent normalement. À la fin des événements, les écrans reviennent à la présentation par défaut. Pour une récurrence quotidienne (par exemple 07:00 - 21:00 en semaine), créez l'événement sur le premier jour puis dupliquez-le, ou programmez les occurrences à l'avance : le calendrier hebdomadaire offre une vue de contrôle immédiate.

> **Bonne pratique**
>
> Réservez la boucle ON aux contenus courts destinés à tourner (annonces, vitrine) et la boucle OFF aux diffusions à passage unique (consignes de sécurité, communiqué ponctuel). Vérifiez toujours le badge Boucle affiché sur l'événement après enregistrement.

11. Le lecteur web et l'application Android

Le lecteur web (/player)

Le lecteur web est la page **http://SERVEUR:3000/player**, à ouvrir en plein écran sur l'écran de diffusion. Après un appairage initial (roue dentée, choix de l'écran ou saisie de la clé), il suit automatiquement la planification : événements, campagnes et présentation par défaut, avec une synchronisation de 60 secondes maximum. Il respecte la boucle ON/OFF de l'événement courant et joue les vidéos — y compris les flux IPTV — via un moteur adapté (HLS / mpegts), et affiche un diagnostic clair avec boutons « Réessayer » et « Lecture directe » quand un contenu refuse (site non intégrable, flux IPTV refusé). La page ne comporte **aucun bouton volume** : le son est piloté uniquement depuis le CMS — case « Démarrer muet » du widget vidéo ou interrupteur « Muet » du widget IPTV dans l'éditeur de présentations — garantissant le même réglage sur tous les écrans, sans contournement possible depuis l'appareil. Le démarrage vidéo est durci : lecture lancée muette si le navigateur l'impose, puis son rétabli conformément au réglage CMS au premier toucher sur la page.



Figure 11 — Lecteur web en diffusion plein écran : page web servie par le proxy intégré, sans aucun bouton volume (nom de l'écran en bas à droite)

Mode kiosk pour écrans dédiés

1. Installez Chrome ou Edge sur le mini-PC ou la box TV reliée à l'écran.

2. Créez un raccourci avec l'option kiosk : **chrome --kiosk http://SERVEUR:3000/player** (l'écran occupe tout l'affichage, sans barre d'outils).
3. Ajoutez ce raccourci au démarrage automatique de la machine pour qu'un redémarrage restaure la diffusion sans intervention.
4. Sur l'écran de veille et l'économiseur d'énergie : désactivez la mise en veille du système dans les réglages du système d'exploitation.

Application Android (APK)

L'application **SignageCMS Player** offre la même expérience sur téléviseurs Android, tablettes et box, avec un moteur natif ExoPlayer/VLC plus robuste pour la vidéo et l'IPTV, un clavier virtuel pour les pages web et la gestion complète des widgets. Le premier lancement demande l'adresse du serveur puis la clé de l'écran ; le dialogue « Connecter Android » du CMS fournit les deux, avec un QR code scannable. Le lecteur vidéo dispose de vrais tampons généreux (réservoir de 20 secondes, re-bufferisation à 5 s puis 10 s) et **relance deux fois** la vidéo avant de basculer sur VLC : une erreur réseau passagère ne dégrade plus la lecture en décodage logiciel. L'audio n'est pas décodé quand le widget est muet, et le décodage matériel est privilégié : les vidéos H.264 et H.265 y sont nettement plus fluides. Des versions précédentes restent servies en secours dans le dossier apk/ du serveur. Après installation, autorisez l'installation depuis des sources inconnues si Android le demande, puis remplacez simplement l'APK pour mettre à jour sans perdre la configuration. L'écran de premier lancement est illustré ci-dessous (figure 12).

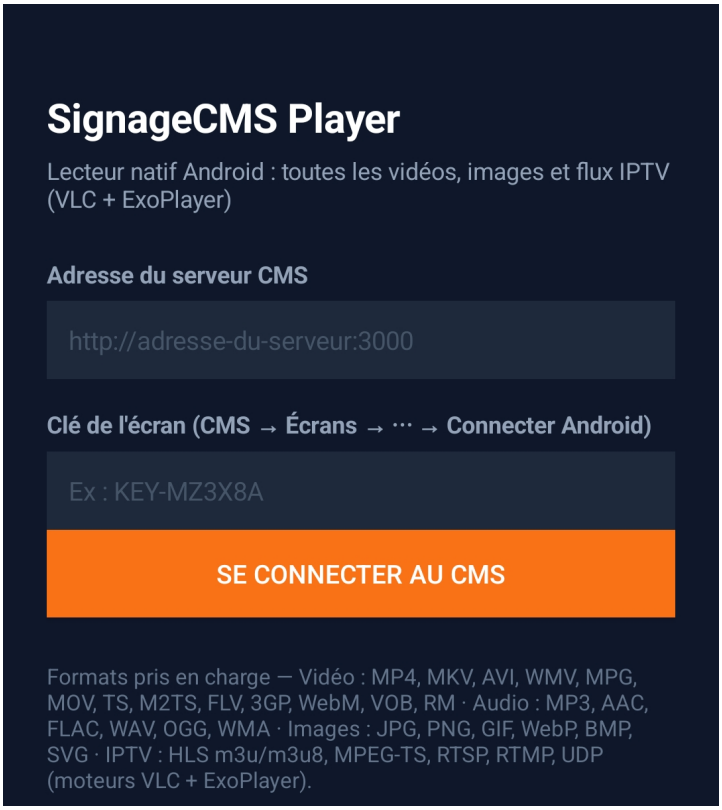


Figure 12 — Application Android SignageCMS Player : premier lancement — adresse du serveur CMS et clé de l'écran, toutes deux fournies par le dialogue « Connecter Android » du CMS

Pages web et TV / IPTV sur les écrans

Le widget **Page web** affiche un site dans une région du lecteur. Les sites qui refusent l'intégration en iframe (en-têtes X-Frame-Options ou CSP frame-ancestors) sont gérés automatiquement : le lecteur bascule seul sur le **proxy de pages intégré** du serveur, et la page reste entièrement interactive (liens, formulaires, connexions). Le proxy n'accepte que des sites publics — les adresses internes sont refusées (garde anti-SSRF). De même, le module **TV / IPTV** diffuse les chaînes de vos comptes Xtream ou listes M3U via un proxy intégré : quand plusieurs écrans regardent la même chaîne, une seule connexion amont est ouverte et redistribuée à tous, avec pool multi-comptes, bascule de secours et timeshift (relecture décalée) optionnel.

Formats de médias et compatibilité des lecteurs

Le tableau ci-dessous récapitule, famille par famille, ce que lit réellement chaque lecteur : l'application Android **SignageCMS Player** (moteurs ExoPlayer + VLC, liste reprise sous le formulaire de connexion — figure 12) et le **lecteur web /player** (décodage du navigateur, complété par hls.js, mpegts.js et dash.js pour les flux). Trois principes expliquent les différences entre les colonnes. D'abord, être **importé** dans la bibliothèque ne garantit pas d'être **lisible** : le serveur accepte certains conteneurs (AVI, MKV, TS…) que le navigateur ne sait pas ensuite décoder. Ensuite, le lecteur web délègue tout au navigateur — seuls les codecs natifs (H.264, VP8/VP9, AAC, MP3…) y passent — tandis que l'APK s'appuie sur VLC, qui absorbe presque tous les conteneurs. Enfin, les documents PowerPoint et PDF ne sont rendus par aucun des deux lecteurs : le serveur les convertit en diapositives images (LibreOffice + Poppler), leur rendu est donc strictement identique partout.

Famille	Extensions	Android	Lecteur web	Remarques
Vidéo	MP4, M4V, MOV (H.264 + AAC)	Oui	Oui	Format recommandé — lecture native sur tous les lecteurs
Vidéo	MP4, MOV (H.265/HEVC)	Oui	Selon navigateur	Décodage matériel côté APK ; côté web, Safari seulement le plus souvent
Vidéo	WebM (VP8/VP9)	Oui	Oui	Natif dans Chrome, Edge et Firefox
Vidéo	MKV	Oui	Selon navigateur	VLC lit tous les MKV ; un navigateur ne garantit pas ce conteneur
Vidéo	AVI, WMV, MPG, VOB, RM	Oui	Non	VLC côté APK ; WMV/VOB/RM refusés à l'import — convertissez en MP4/H.264
Vidéo	TS, M2TS, FLV, 3GP	Oui	Non	M2TS et FLV refusés à l'import ; en flux IPTV, TS et FLV restent lisibles côté web (mpegts.js)
Audio	MP3, AAC, M4A, WAV	Oui	Oui	Lecture native dans les navigateurs
Audio	OGG, FLAC	Oui	Oui	Chrome, Edge, Firefox ; FLAC aussi sur Safari 11

Famille	Extensions	Android	Lecteur web	Remarques
Audio	WMA	Oui	Non	VLC côté APK ; non importable dans la bibliothèque — convertissez en MP3
Image	JPG, PNG, GIF	Oui	Oui	Y compris GIF animé
Image	WebP, BMP, SVG	Oui	Oui	Rendu identique sur les deux lecteurs
Image	ICO, TIFF	Non	ICO oui · TIFF non	TIFF n'est décodé ni par Android ni par les navigateurs — préférez PNG/JPG
Document	PPT, PPTX, PPS, PPSX, ODP	Oui	Oui	Convertis en diapositives images par le serveur (LibreOffice) — rendu identique partout
Document	PDF	Oui	Oui	Converti en diapositives images (Poppler) côté serveur
Page web	URL http(s)	Oui	Oui	Iframe + proxy de pages intégré ; APK : WebView native avec clavier virtuel
IPTV	HLS (.m3u / .m3u8)	Oui	Oui	Bibliothèque hls.js côté web
IPTV	MPEG-TS, FLV	Oui	Oui	mpegts.js côté web (direct et timeshift via le hub)
IPTV	DASH (.mpd)	Non listé	Oui	dash.js côté web ; absent de la liste affichée par l'APK
IPTV	RTSP, RTMP, UDP	Oui	Non	Protocoles hors navigateur — utilisez l'APK (moteur VLC)
Données	JSON, CSV, XLSX, XLS	Non diffusé	Non diffusé	Jeux de données pour les rapports, pas des médias d'affichage

En pratique

Pour une diffusion sans surprise sur les deux types de lecteurs, encodez vos vidéos en **MP4 (H.264 + AAC)** et vos images en JPG ou PNG. Quand le navigateur ne sait pas lire un fichier, le widget vidéo affiche « Format vidéo non lisible par ce navigateur » et rappelle le recours : le fichier reste lisible sur le lecteur Android (VLC) ou se convertit en MP4/H.264 avec un outil comme ffmpeg ou HandBrake. Côté audio, WMA est la seule exception de la liste (VLC seulement) ; côté images, évitez TIFF et ICO au profit de JPG/PNG.

12. Maintenance, sécurité et dépannage

Sauvegarde et restauration

Toute la configuration tient dans un fichier unique : **db/custom.db** (comptes, planifications, réglages IPTV et LDAP, références des médias). Une copie à chaud de ce fichier est acceptée — planifiez une copie quotidienne vers un disque ou un partage réseau. Pensez également à sauvegarder le dossier **public/uploads** qui contient les fichiers médias eux-mêmes. La restauration consiste à remettre ces deux éléments en place puis à redémarrer le service : le serveur retrouve exactement son état.

Mise à jour du serveur

1. Téléchargez la nouvelle archive serveur et sauvegardez db/custom.db ainsi que public/uploads.
2. Remplacez les fichiers du projet (en conservant vos dossiers db et public/uploads), puis exécutez npm install.
3. Recompilez : npm run build (ou deploy\windows\build.bat sous Windows).
4. Redémarrez le service (systemctl restart signagecms, ou start-cms.bat) et vérifiez http://localhost:3000/api/version.

Réflexes de sécurité au quotidien

- Changez le mot de passe admin initial dès la première connexion et supprimez db/admin-credentials.txt.
- Utilisez des comptes nominatifs avec le rôle le plus restrictif possible et la matrice de permissions plutôt que de partager le compte admin.
- Activez HTTPS dès que le CMS sort du réseau local : le Caddyfile fourni permet un reverse proxy TLS en deux lignes.
- N'exposez pas le port 3000 directement sur Internet sans reverse proxy avec limitation de débit.
- Après une intrusion suspectée, changez tous les mots de passe : chaque réinitialisation révoque immédiatement les sessions du compte.

Dépannage des incidents courants

Symptôme	Cause probable	Solution
« Port déjà utilisé » au démarrage	Autre service occupe le 3000	Démarrez avec un autre port : set PORT=8080 (Windows) ou PORT=8080 (Linux), adaptez le pare-feu
Écran inaccessible depuis le réseau	Pare-feu fermé ou règle absente	Windows : firewall.ps1 en administrateur. Linux : ufw allow 3000/tcp. Vérifiez l'IP du serveur (ipconfig / ip addr)

Symptôme	Cause probable	Solution
Conversion PPT/PDF absente	LibreOffice ou Poppler manquants	install-tools.bat (Windows, admin) ou sudo bash deploy/linux/install-tools.sh ; puis Reconvertir dans la bibliothèque
« Trop de tentatives échouées »	Limiteur anti-force-brute déclenché	Attendez 15 minutes ou redémarrez le serveur ; vérifiez la casse du clavier
Site web « inaccessible » dans le lecteur	Site anti-robot ou redirection JS cross-origin	Bouton Réessayer ; utilisez l'URL exacte du contenu ; l'APK Android s'affranchit de ces limites (WebView natif)
Vidéo qui ne démarre pas ou reste figée au chargement	Serveur lancé avec Bun ou sans support des requêtes Range	Lancez le serveur avec Node.js (start.sh ou node server.js) : les lecteurs vidéo, web et APK, exigent des réponses 206 aux requêtes Range sur les médias
Flux IPTV « quota atteint »	Tous les slots du fournisseur sont occupés	Le pool bascule seul sur les autres comptes ; sinon réduisez les écrans simultanés ou ajoutez un compte
Un écran reste hors ligne	Lecteur fermé, machine en veille ou clé erronée	Vérifiez le lecteur sur l'écran, la clé (menu ... de l'écran) et le réseau ; le journal d'activité montre le dernier accès
Mot de passe admin perdu	Aucun autre compte admin	Supprimez le compte admin de la base puis relancez : nouveau mot de passe aléatoire (console + db/admin-credentials.txt)
Le build échoue	Node.js inférieur à 20.9	Installez Node.js 20/22 LTS puis relancez build.bat (journal : build-cms.log)
Service Windows instable	Conflit de port ou chemin déplacé	Consultez service.log à la racine, réinstallez avec install-service.ps1 après correction

Pour aller plus loin dans le diagnostic, trois sources complètent ce tableau : le **journal d'activité** du tableau de bord et le module **Rapports** (journaux serveur filtrables par niveau et par écran), les fichiers journaux du projet (server.log, dev.log, service.log, install-cms.log, build-cms.log) et la page **/api/version** qui confirme en un appel la version exacte du serveur en cours d'exécution. En cas de comportement inexpliqué d'un lecteur, un simple rechargement de la page du lecteur web (ou un redémarrage de l'application Android) provoque une resynchronisation complète avec le serveur.

Annexe A — Référence rapide

Liens de téléchargement

Les fichiers d'installation sont servis directement par votre instance SignageCMS de démonstration (remplacez le domaine par l'adresse de votre propre serveur pour une autre installation). Ces liens restent valides en permanence.

```
Archive du serveur CMS (zip, inclut install.sh et start.sh) :
https://preview-chat-3bdaeb01-bf73-432b-93a3-0c035f5aa616.space-z.ai/downloads/SignageCMS-Serveur.zip

Script install.sh seul (installation complete Linux) :
https://preview-chat-3bdaeb01-bf73-432b-93a3-0c035f5aa616.space-z.ai/downloads/install.sh

Script start.sh seul (lancement du serveur) :
https://preview-chat-3bdaeb01-bf73-432b-93a3-0c035f5aa616.space-z.ai/downloads/start.sh

Application Android SignageCMS Player (la plus recente) :
https://preview-chat-3bdaeb01-bf73-432b-93a3-0c035f5aa616.space-z.ai/apk/SignageCMS-Player.apk
```

Variables d'environnement (.env)

Variable	Rôle	Défaut
DATABASE_URL	Chemin absolu de la base SQLite (file:C:/SignageCMS/db/custom.db)	Créé par install.bat
PORT	Port d'écoute du serveur	3000
MAX_UPLOAD_MB	Taille maximale d'un média importé (Mo)	50 ou valeur Paramètres
ADMIN_DEFAULT_PASSWORD	Impose le mot de passe initial du compte admin (installations scriptées)	Aléatoire
ALLOW_FRAME_EMBED	Autorise l'intégration de l'interface CMS dans une iframe externe (1 = activé)	Désactivé
SOFFICE_PATH	Chemin explicite de soffice (conversion PPT)	Détection automatique
PDFTOPPM_PATH	Chemin explicite de pdftoppm (conversion PDF)	Détection automatique

Fichiers et dossiers importants

Chemin	Contenu
db/custom.db	Base SQLite : comptes, planifications, réglages, références médias. À sauvegarder.
db/admin-credentials.txt	Mot de passe admin initial — à supprimer après la première connexion.
public/uploads/	Fichiers médias servis aux lecteurs. À sauvegarder avec la base.
deploy/windows/	Scripts d'installation et de service Windows (install, start, firewall, service).
deploy/linux/	install.sh (installation complete), start.sh (lancement), install-tools.sh (outils seuls), LISEZMOI.txt.
server.log / service.log	Journaux du serveur en production et du service Windows.
.env	Configuration locale (base, port, limites d'upload, chemins des outils).

URLs utiles du serveur

URL	Usage
http://SERVEUR:3000/	Interface d'administration (connexion par compte).
http://SERVEUR:3000/player	Lecteur web plein écran (appairage par roue dentée ou clé).
http://SERVEUR:3000/api/version	Version exacte du serveur en cours d'exécution.
http://SERVEUR:3000/downloads/	Téléchargement de l'archive serveur.
http://SERVEUR:3000/apk/	Téléchargement des applications Android.